Table of Contents

课程简介 0

Flask介绍 1

工程搭建 2

- 环境安装 2.1
- HelloWorld程序 2.2
- 参数说明 2.3
- 开发服务器运行方式 2.4

路由与蓝图 3

- 路由 3.1
- 蓝图 3.2

请求与响应 4

- 处理请求 4.1
- 处理响应 4.2
- Cookie与Session 4.3

请求钩子与上下文 5

- 异常处理 5.1
- 请求钩子 5.2
- 上下文 5.3

Flask-RESTful 6

- 快速认识FLASK-RESTful 6.1
- 关于视图 6.2
- 关于请求 6.3
- 关于响应 6.4

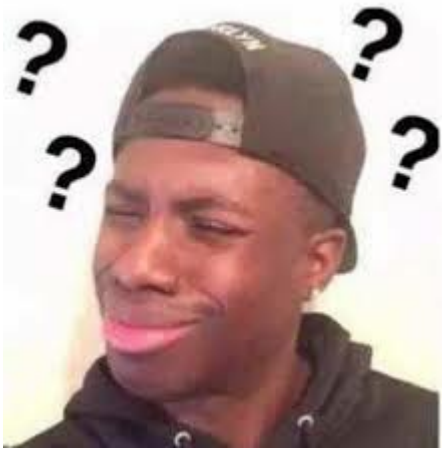
课程简介

[TOC]

- 1. 学习目标
  - 1.1 思考
  - 1.2 明确几点思想
  - 1.3 学习目标
- 学习方法
- 课程内容
- 框架学习目标

1. 学习目标

1.1 思考

> 为什么学习框架/工具？
>
> 
>

1.2 明确几点思想

> IT哲学—挨踢哲学：
>
> - 先有问题/需求，再有解决工具，工具是为解决问题/需求而诞生的。**
>
> 故，学习重点是探究并理解问题的内在本质，及解决问题的思想和思路。
>
> - 可能没有最好的工具，只有更合适的工具。

1.3 学习目标

> - 明确问题/需求及其背景（即工具的应用场景）

> - 理解并掌握解决问题的思想和思路
> - 理解后熟练运用工具

学习方法

> - 不机械的抄代码、背代码，锻炼自己写代码的能力
> - 先思考 再动手
>
> 先想清问题是什么，再想清解决思路，最后动手将思路实现
> - 多思考 勤动手
>
> 多问自己为什么，多提出不同的解决方式再动手验证可行性，多调bug

课程内容

> - Flask框架
> - Flask-RESTful 扩展

框架学习目标

> 1. 如何编写试图
> 2. 如何处理请求
> 3. 如何构造响应

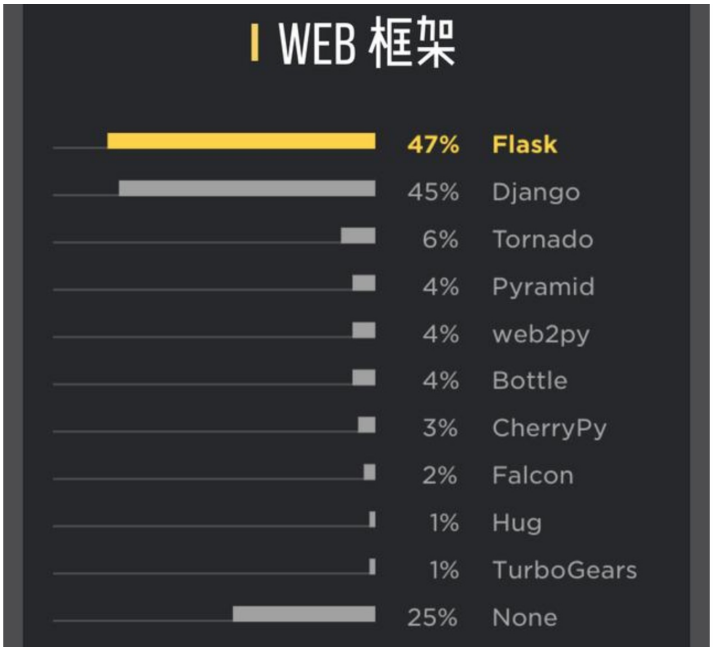
toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 17:37:52

Flask介绍

[TOC]

- 1 学习Flask框架的原因
- 2 Flask
- 3 框架对比
- 4 常用扩展包
- 5 Flask文档

1 学习Flask框架的原因

> 2018 Python官方年度调研报告显示Flask与Django为Python Web开发使用最主要的两个框架。
>
> 
>

2 Flask

> Flask诞生于2010年，是Armin ronacher（人名）用 Python 语言基于 Werkzeug 工具箱编写的轻量级Web开发框架。

> Flask 本身相当于一个内核，其他几乎所有的功能都要用到扩展（邮件扩展Flask-Mail，用户认证Flask-Login，数据库Flask-SQLAlchemy），都需要用第三方的扩展来实现。比如可以用 Flask 扩展加入ORM、窗体验证工具，文件上传、身份验证等。Flask 没有默认使用的数据库，你可以选择 MySQL，也可以用 NoSQL。
>
> 其 WSGI 工具箱采用 Werkzeug（路由模块），模板引擎则使用 Jinja2。这两个也是 Flask 框架的核心。
>
> 最新版本 **1.0.2**

3 框架对比

> - 框架轻重
> - 重量级的框架：为方便业务程序的开发，提供了丰富的工具、组件，如Django
> - 轻量级的框架：只提供Web框架的核心功能，自由、灵活、高度定制，如Flask、Tornado
> - 与Django对比,django提供了：
> > django-admin快速创建项目工程目录
> >
> > manage.py 管理项目工程
> >
> > orm模型（数据库抽象层）
> >
> > admin后台管理站点
> >
> > 缓存机制
> >
> > 文件存储系统
> >
> > 用户认证系统
> - 而这些，flask都没有，都需要扩展包来提供

4 常用扩展包

> 扩展列表：http://flask.pocoo.org/extensions/
>
> - Flask-SQLalchemy：操作数据库；
> - Flask-script：插入脚本；
> - Flask-migrate：管理迁移数据库；
> - Flask-Session：Session存储方式指定；
> - Flask-WTF：表单；
> - Flask-Mail：邮件；
> - Flask-Bable：提供国际化和本地化支持，翻译；
> - Flask-Login：认证用户状态；

> - Flask-OpenID：认证；
> - **Flask-RESTful**：开发**REST API**的工具；
> - Flask-Bootstrap：集成前端Twitter Bootstrap框架；
> - Flask-Moment：本地化日期和时间；
> - Flask-Admin：简单而可扩展的管理接口的框架

5 Flask文档

> - 中文文档（http://docs.jinkan.org/docs/flask/）
> - 英文文档（http://flask.pocoo.org/docs/1.0/）

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 17:26:33

工程搭建

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-05-07 22:21:31

环境安装

1. 复习虚拟环境和pip的命令

> ```
> # 虚拟环境
> mkvirtualenv  # 创建虚拟环境
> rmvirtualenv  # 删除虚拟环境
> workon  # 进入虚拟环境、查看所有虚拟环境
> source /[path]/env_name/bin/activate # 或者用source来切入虚拟环境
> deactivate  # 退出虚拟环境
>
> # pip
> pip install  # 安装依赖包
> pip uninstall  # 卸载依赖包
> pip list  # 查看已安装的依赖包
> pip freeze  # 冻结当前环境的依赖包
> ```

2. 创建虚拟环境

> 注意需要联网
>
> ```
> whereis python # 查看python解释器执行文件的路径
> mkvirtualenv venv_name --python=/usr/bin/python3.6 # 指定python版本来创建虚拟环境
> ```

3. 安装Flask

> 默认安装flask最新版本，注意需要联网
>
> ```
> pip install flask
> ```

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 17:36:58

Flask的HelloWorld

[TOC]

- 1. Flask程序编写
- 2. 启动运行

1. Flask程序编写

> 创建helloworld.py文件
>
> ```
> # 导入Flask类
> from flask import Flask
>
> #Flask类接收一个参数__name__
> app = Flask(__name__)
>
> # 装饰器的作用是将路由映射到视图函数index
> @app.route('/')
> def index():
>  return 'Hello World'
>
> # Flask应用程序实例的run方法启动WEB服务器
> if __name__ == '__main__':
>  app.run()
> ```

2. 启动运行

> - 命令行终端用python解释器运行
>
> > 切换至虚拟环境中执行
>
>  ```
>  workon venv_name
>  python helloworld.py
>  ```
>
> - pycharm 运行
>
> 像正常运行普通python程序一样即可。
>
> - 命令行终端用flask命令运行
>
>  ```
>  workon venv_name # 切入虚拟环境
>  export FLASK_APP=helloworld # 设置环境变量
>  flask run -h 0.0.0.0 -p 5000 # 启动
>  ```

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 17:44:24

参数说明

> - Flask对象的初始化参数
> - 应用程序配置参数
> - app.run()运行参数

[TOC]

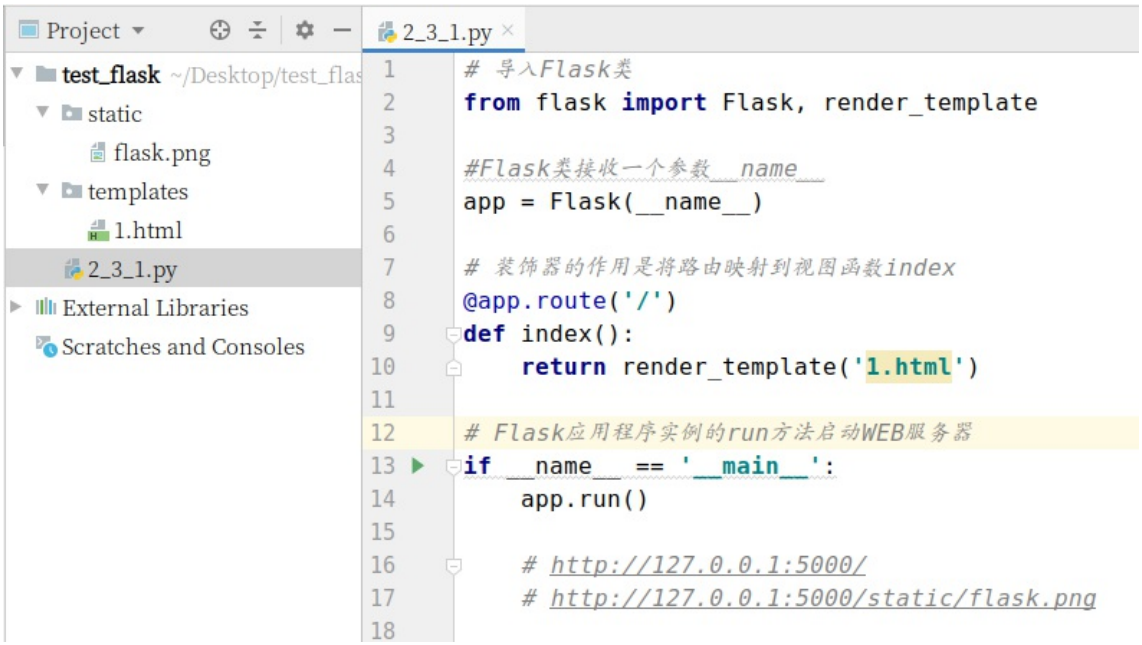
- 1. Flask对象初始化参数
  - 1.1 默认参数情况下的项目路径
  - 1.2 修改参数的情况下
- 2. 应用程序配置参数
  - 2.1 从配置类中加载
  - 2.2 从配置文件中加载
  - 2.3 从环境变量中加载
- 3. app.run 参数
- 4. 项目中的常用工厂模式创建Flask_app

1. Flask对象初始化参数

> Flask 程序实例在创建的时候，需要默认传入当前 Flask 程序所指定的包(模块)，接下来就来详细查看一下 Flask 应用程序在创建的时候一些需要我们关注的参数

1.1 默认参数情况下的项目路径

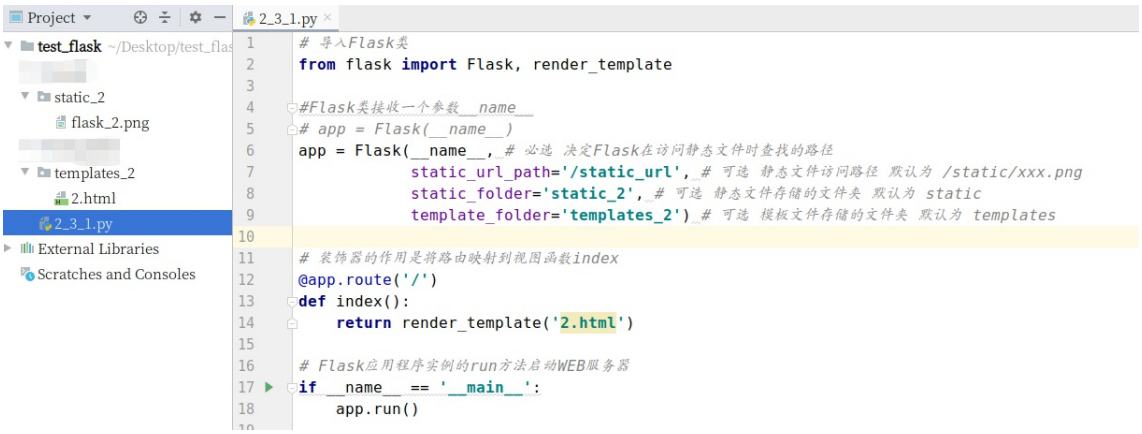
> http://127.0.0.1:5000/
>
> http://127.0.0.1:5000/static/flask.png



1.2 修改参数的情况下

http://127.0.0.1:5000/

http://127.0.0.1:5000/static_url/flask_2.png



2. 应用程序配置参数

> 对于Flask对象初始化参数仅仅设置的是Flask本身的属性，比如：
>
> - Flask从哪里读取静态文件
> - Flask从哪里读取模板文件
> - ...
>
> 应用程序配置参数设置的是一个Web应用工程的相关信息，比如：
>
> - 数据库的连接信息
> - 日志的配置信息
> - 自定义的配置信息

> - ...
>
> - 配置作用
> - 集中管理项目的所有配置信息
> - 使用方式
>
> > Django将所有配置信息都放到了settings.py文件中，而Flask则不同。
>
> **Flask**将配置信息保存到了 `app.config` 属性中，该属性可以按照字典类型进行操作。
>
> - 读取
> - `app.config.get(name)`
> - `app.config[name]`
> - 设置
>
> > 有三种方式：
> >
> > - 从配置对象中加载
> > - 从配置文件中加载
> > - 从环境变量中加载

2.1 从配置类中加载

> `app.config.from_object(配置类)`
>
> 应用场景：作为默认配置写在程序代码中
>
> ```
> from flask import Flask
> app = Flask(__name__)
> # 配置类
> class DefaultConfig(object):
>  """默认配置"""
>  SECRET_KEY = 'TPmi4aLWRbyVq8zu9v82dWYW1'
> # 从配置对象中加载
> app.config.from_object(DefaultConfig)
>
> @app.route("/")
> def index():
>  print(app.config['SECRET_KEY'])
>  return app.config['SECRET_KEY']
>
> if __name__ == '__main__':
>  app.run()
> ```
>
> - 可以继承
>
> ```
> from flask import Flask
> ```

```
app = Flask(__name__)
# 配置类
class DefaultConfig(object):
    """默认配置"""
    SECRET_KEY = 'TPmi4aLWRbyVq8zu9v82dWYW1'
# 发生继承的配置类
class DevelopmentConfig(DefaultConfig):
    DEBUG=True
    SECRET_KEY_2 = '这是2'
# 从发生继承的配置类中加载
app.config.from_object(DevelopmentConfig)

@app.route("/")
def index():
    print(app.config['SECRET_KEY'])
    print(app.config['SECRET_KEY_2'])
    return app.config['SECRET_KEY_2']

if __name__ == '__main__':
    app.run()
```

2.2 从配置文件中加载

`app.config.from_pyfile(配置文件)`

应用场景：在项目中使用固定的配置文件

- 新建一个配置文件settings.py

  ```
  SECRET_KEY = '这是settings.py中的SECRET_KEY'
  ```

- 在Flask程序文件中

  ```
  from flask import Flask
  app = Flask(__name__)
  # 从配置文件中加载配置文件，文件中所有变量就加载到app.config字典中
  app.config.from_pyfile('settings.py')

  @app.route("/")
  def index():
      print(app.config['SECRET_KEY'])
      return app.config['SECRET_KEY']

  if __name__ == '__main__':
      app.run()
  ```

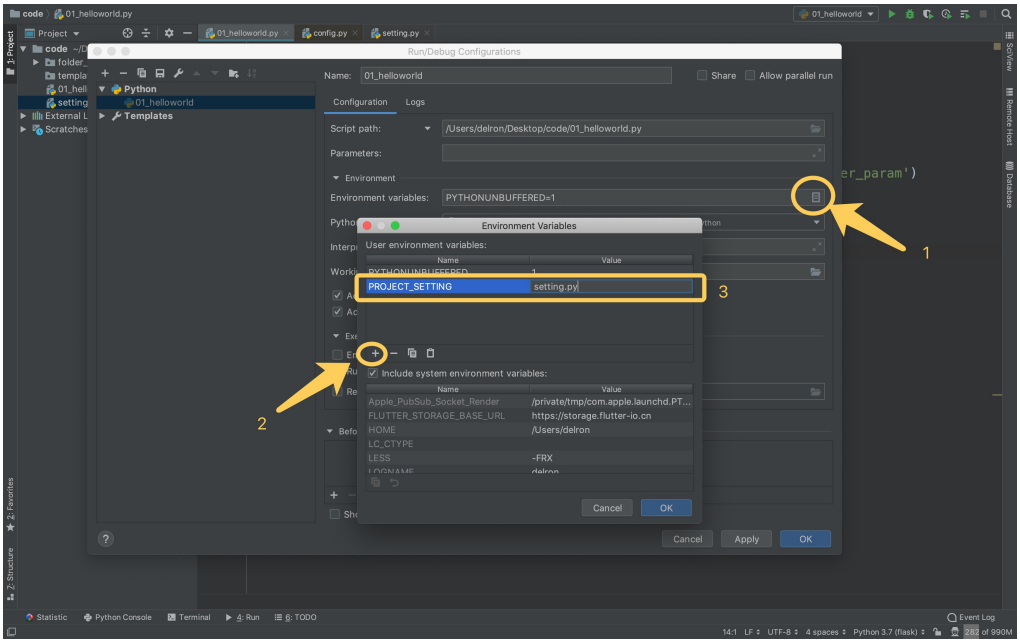
2.3 从环境变量中加载

> 环境变量（environment variables）一般是指在操作系统中用来指定操作系统运行环境的一些参数，如：临时文件夹位置和系统文件夹位置等。 环境变量是在操作系统中一个具有特定名字的对象，它包含了一个或者多个应用程序所将使用到的信息。
>
> > 通俗的理解，环境变量就是我们设置在操作系统中，由操作系统代为保存的变量值
>
> - 在Linux系统中设置和读取环境变量的方式如下：
>
> ```
> export ITCAST=python # 设置 export 变量名=变量值
> echo $ITCAST # 读取 echo $变量名
> ```
>
> - 在flask中从环境变量中加载配置
>
> > **Flask**使用环境变量加载配置的本质是通过环境变量值找到配置文件，再读取配置文件的信息.
> >
> > 其使用方式为 `app.config.from_envvar('环境变量名')` ，该 `环境变量名` 值为 `配置文件的绝对路径`
> >
> > - 应用场景：
> > - 配置文件的地址不固定；
> > - 在代码中不想暴露真实的配置文件地址，只在运行代码的服务器上才有真实配置文件的信息。
>
> - 先在终端中执行如下命令
>
> > ```
> > export PROJECT_SETTING='.../settings.py'
> > # /home/worker/Desktop/test_flask/settings.py
> > # 不能~/Desktop/test_flask/settings.py
> > ```
>
> - 再运行如下代码，运行方式：在终端中**python xxx.py**运行！
>
> > ```
> > from flask import Flask
> > app = Flask(__name__)
> >
> > # silent=True 安静的处理，即时没有值也让Flask正常的运行下去
> > app.config.from_envvar('PROJECT_SETTING', silent=True)
> >
> > @app.route("/")
> > def index():
> > print(app.config['SECRET_KEY'])
> > return app.config['SECRET_KEY']
> >
> > if __name__ == '__main__':
> > app.run()
> > ```

- 关于 `silent` 参数的说明：

  > 表示系统环境变量中没有设置相应值时是否抛出异常
  >
  > - False 表示不安静的处理，没有值时报错通知，默认为False
  > - True 表示安静的处理，即时没有值也让Flask正常的运行下去

- Pycharm运行时设置环境变量的方式

  > 按下图设置完毕后，点击 `Edit Configurations...` 右边的绿色运行按钮
  >
  >
  >
  >
  > 
  >

3. app.run 参数

> 可以指定运行的主机IP地址，端口，是否开启调试模式
>
> ```
> ......
> if __name__ == '__main__':
>  app.run(host="0.0.0.0", port=5000, debug=True)
> ```
>
> 关于DEBUG调试模式
>
> - 程序代码修改后可以自动重启服务器

> - 在服务器出现相关错误的时候可以直接将错误信息返回到前端进行展示

4. 项目中的常用工厂模式创建Flask_app

> 使用工厂模式创建Flask app，并结合使用配置对象与环境变量加载配置
>
> - 使用配置对象加载默认配置
> - 使用环境变量加载不想出现在代码中的敏感配置信息
>
> ```
> from flask import Flask
>
> class DefaultConfig(object):
>  """默认配置"""
>  SECRET_KEY = 'itcast1'
>
> class DevelopmentConfig(DefaultConfig):
>  DEBUG = True # 相当于app.run(debug=True)
>
> # 创建flask app的函数
> def create_flask_app(config):
>  """
>  创建Flask应用
>  :param config: 配置类
>  :return: Flask应用对象
>  """
>  app = Flask(__name__)
>  # 从配置类加载配置信息
>  app.config.from_object(config)
>
>  # 从环境变量指向的配置文件中读取的配置信息会覆盖掉从配置对象中加载的同名参数
>  # app.config.from_envvar("PROJECT_SETTING", silent=True)
>  return app
>
> # 创建flask app
> app = create_flask_app(DevelopmentConfig)
>
> @app.route("/")
> def index():
>  print(app.config['SECRET_KEY'])
>  return "hello world"
>
> if __name__ == '__main__':
>  app.run()
> ```

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 17:50:12

开发服务器启动方式

> 在flask1.0版本之后，Flask新增了flask app的启动方式，在命令行中输入 `flask run` 启动。
>
> ```
> from flask import Flask
> app = Flask(__name__)
>
> @app.route('/')
> def index():
>  return 'Hello World'
>
> # 程序中可以不用再写app.run()
> # 在终端中设置FLASK_APP环境变量后
> # 以 flask run -p port 启动程序
> ```

[TOC]

- 1. 终端启动
  - 1.1 说明
  - 1.2 也可以以 python -m flask run 的方式运行
- 2. Pycharm启动
  - 2.1 新版pycharm设置
  - 2.2 旧版本Pycharm设置

1. 终端启动

> ```
> export FLASK_APP='./xxx.py'
> flask run -h 0.0.0.0 -p 5000
> ```

1.1 说明

> - 环境变量 FLASK_APP 指明flask的启动实例
> - `flask run -h 0.0.0.0 -p 5000` 绑定地址 端口
> - `flask run --help` 获取帮助
> - 生产模式与开发模式的控制
>
> 通过 `FLASK_ENV` 环境变量指明
>
> - `export FLASK_ENV=production` 运行在生产模式，未指明则默认为此方式
> - `export FLASK_ENV=development` 运行在开发模式

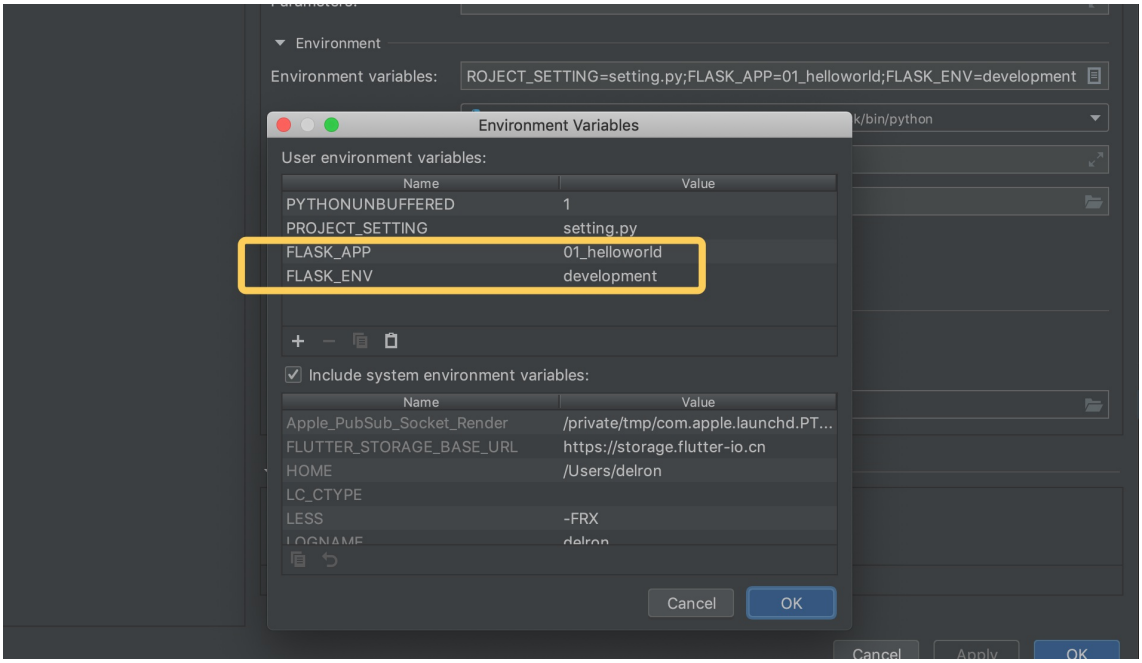
1.2 也可以以 python -m flask run 的方式运行

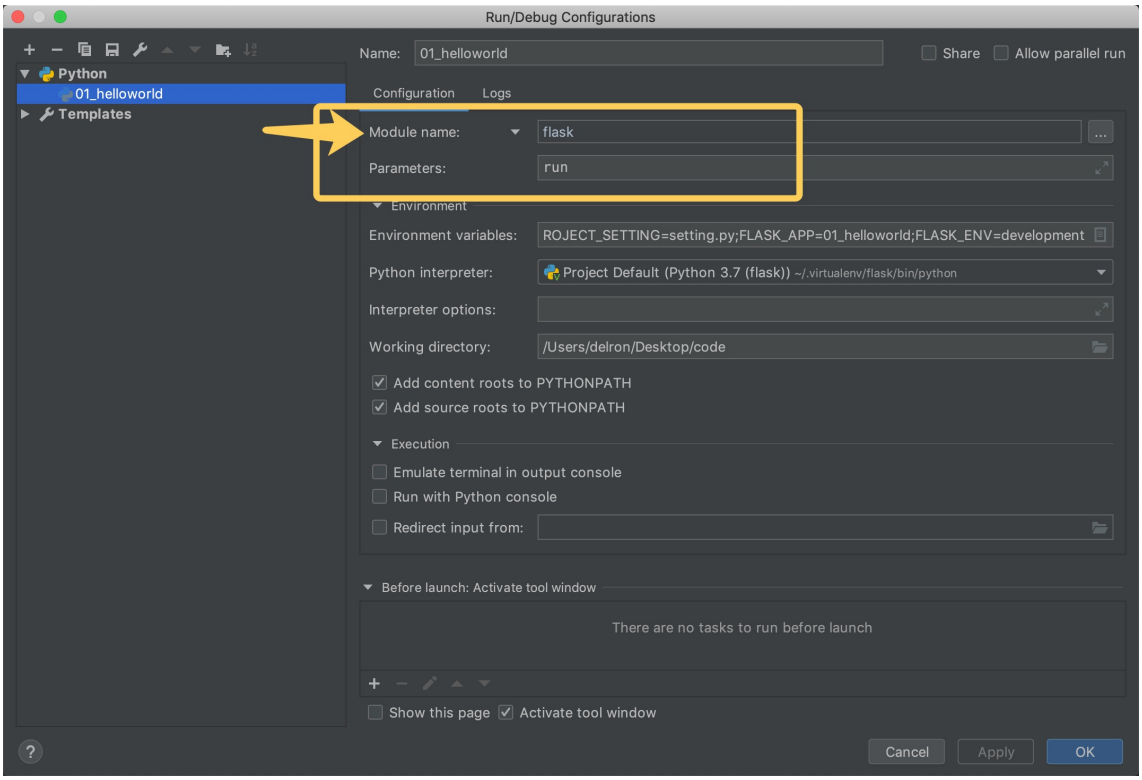
```
export FLASK_APP=helloworld
python -m flask run
```

2. Pycharm启动

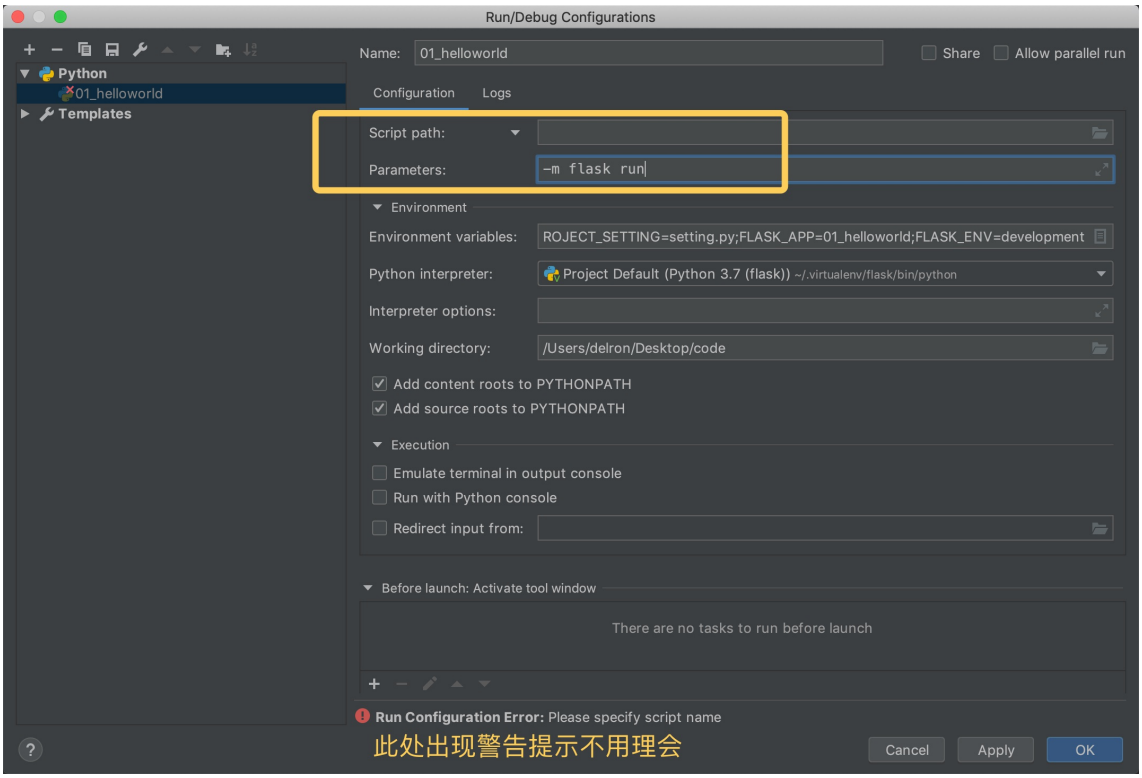
2.1 新版pycharm设置

> 如下图设置好环境变量后，就可以点击运行按钮了





2.2 旧版本Pycharm设置



toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 17:52:25

路由与蓝图

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-05-07 22:21:37

路由

```
@app.route("/itcast")
def view_func():
    return "hello world"
```

[TOC]

- 1. 查询路由信息
  - 1.1 命令行方式
  - 1.2 在程序中获取
  - 1.3 练习
- 2. 指定请求方式

1. 查询路由信息

1.1 命令行方式

```
export FLASK_APP=helloworld
flask routes
# 输出类似下面的信息
Endpoint  Methods  Rule
--------  -------  -----------------------
index     GET      /
static    GET      /static/
```

1.2 在程序中获取

> - 在应用（实例化的flask app）中的url_map属性中保存着整个Flask应用的路由映射信息，可以通过读取这个属性获取路由信息
>
> ```
> print(app.url_map)
> ```
>
> - 如果想在程序中遍历路由信息，可以采用如下方式
>
> ```
> for rule in app.url_map.iter_rules():
>  print('name={} path={}'.format(rule.endpoint, rule.rule))
> ```

1.3 练习

- 需求
  - 通过访问 `/` 地址，以json的方式返回应用内的所有路由信息
- 实现

```
import json
from flask import Flask
app = Flask(__name__)

@app.route('/')
def route_map():
 """返回所有视图网址"""
 rules_iterator = app.url_map.iter_rules()
 return json.dumps({rule.endpoint: rule.rule for rule in rules_iterator})

if __name__ == '__main__':
 app.run()
```

2. 指定请求方式

- 在 Flask 中，如果不显式声明请求方式，其默认的请求方式有：
  - GET
  - OPTIONS(自带)
  - HEAD(自带)
- 利用 `methods` 参数可以自己指定一个接口的请求方式

```
from flask import Flask, request
app = Flask(__name__)

@app.route("/itcast1", methods=["POST"])
def view_func_1():
   return "hello world 1"
# methods参数接收一个list，请求方式为大写字符，可以有多个请求方式
@app.route("/itcast2", methods=["GET", "POST"])
def view_func_2():
   # 利用flask.request.method判断是哪种请求方式
   if request.method == 'GET':
       return "hello world 2, get"
   if request.method == 'POST':
       return "hello world 2, post"

if __name__ == '__main__':
   app.run()
```

- 利用requests模块测试上述代码

  > `pip install requests` 安装 `requests` 模块
  >
  > ```
  > import requests
  >
  > url1 = 'http://127.0.0.1:5000/itcast1'
  > resp = requests.head(url1)
  > print(resp.__dict__)
  > resp = requests.options(url1)
  > print(resp.__dict__)
  > resp = requests.post(url1)
  > print(resp.text)
  >
  > url2 = 'http://127.0.0.1:5000/itcast2'
  > resp = requests.post(url2)
  > print(resp.text)
  > resp = requests.get(url2)
  > print(resp.text)
  > ```

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 17:55:31

蓝图

> - 思考：
    - 在一个Flask 应用项目中，如果业务视图过多，可否将以某种方式划分出的业务单元单独维护，将每个单元用到的视图、静态文件、模板文件等独立分开？
    - 例如从业务角度上，可将整个应用划分为用户模块单元、商品模块单元、订单模块单元，如何分别开发这些不同单元，并最终整合到一个项目应用中？
> - 蓝图从项目代码结构上解决了多人合作，共同开发的问题

[TOC]

- 1. 使用蓝图
- 2. 使用方式
    - 2.1 单文件蓝图
    - 2.2 目录（包）蓝图
        - 2.2.1 `main.py`
        - 2.2.2 `user/__init__.py`
        - 2.2.3 `user/user_views.py`
        - 2.2.4 `goods/__init__.py`
        - 2.2.5 `goods/goods_views.py`
- 3. 蓝图中自定义参数
    - 3.1 指定蓝图的url前缀
    - 3.2 指定蓝图内部静态文件夹
        - 3.2.1 注册蓝图的静态目录
        - 3.2.2 设置蓝图静态文件的访问路径
    - 3.3 蓝图内部模板目录
    - 3.4 拓展1:蓝图自定义名称的作用
    - 3.5 拓展2:蓝图设置请求钩子,只会监听该蓝图注册的路由

1. 使用蓝图

> 在Flask中，使用蓝图Blueprint来分模块来进行组织管理。
>
> - 蓝图实际可以理解为是一个存储一组视图方法的容器对象，其具有如下特点：
    - 一个flask_app应用可以具有多个Blueprint
    - 可以将一个Blueprint注册到任何一个未使用的URL下比如 “/user”、“/goods”
    - Blueprint可以单独具有自己的模板、静态文件或者其它的通用操作方法
    - 在一个应用初始化时，就应该要注册需要使用的Blueprint
    - 但是一个Blueprint并不是一个完整的应用，它不能独立于应用运行，而必须要注册到某一个应用中。

2. 使用方式

> 使用蓝图可以分为三个步骤
>
> 1. 创建一个蓝图对象
> 2. 在这个蓝图对象上进行操作，如：注册路由,指定静态文件夹,注册模版过滤器等
> 3. 在应用对象上注册这个蓝图对象
>
> ```
> from flask import Flask, Blueprint
>
> # step 1. 创建一个蓝图对象
> user_bp = Blueprint('user', __name__) # user为蓝图的自定义名字
>
> # step 2. 定义一个蓝图对象的视图函数
> @user_bp.route('/blue')
> def user_profile():
>  return 'this is blueprint'
>
> # step 3. 实例化flask app，并注册蓝图对象
> app = Flask(__name__)
> app.register_blueprint(user_bp) # 注册蓝图对象
>
> @app.route('/')
> def route_map():
>  return 'this is index'
>
> if __name__ == '__main__':
>  app.run(debug=True)
> ```

2.1 单文件蓝图

> 如上面的代码，可以将创建蓝图对象与定义视图放到一个文件中 。

2.2 目录（包）蓝图

> 对于一个打算包含多个文件的蓝图，通常将创建蓝图对象放到Python包的 `__init__.py` 文件中
>
> ```
> ---- project # 工程目录
>  |---- main.py # 启动文件
>  |---- user  #用户蓝图文件夹
>  |      |--- __init__.py  # 此处创建蓝图对象
>  |      |--- user_views.py  # 具体的视图函数所在文件
>  |      |--- ...
>  |
>  |---- goods # 商品蓝图文件夹
> ```

```
            |       |--- __init__.py
            |       |--- goods_views.py  # 具体的视图函数所在文件
            |       |--- ...
            |...
```

> 接下来我们就按照上边的工程目录来实践一下，来切身感受一下蓝图的效果

2.2.1 `main.py`

```
from flask import Flask
  from user import user_bp
  from goods import goods_bp

  app = Flask(__name__)
  app.register_blueprint(user_bp)
  app.register_blueprint(goods_bp)

  @app.route("/")
  def index():
     return "index"

   if __name__ == '__main__':
     app.run(debug=True)
```

2.2.2 `user/__init__.py`

```
from flask import Blueprint
  user_bp = Blueprint('user', __name__)
  from user import user_views # 思考这行代码的作用
```

2.2.3 `user/user_views.py`

```
from user import user_bp
  @user_bp.route('/user')
  def user():
      return 'user'
```

2.2.4 `goods/__init__.py`

```
from flask import Blueprint
  goods_bp = Blueprint('goods', __name__)
  from goods import goods_views # 又来了...
```

2.2.5 `goods/goods_views.py`

```
from goods import goods_bp
  @goods_bp.route('/goods')
  def goods():
      return 'goods'
```

3. 蓝图中自定义参数

> - 定义url前缀
> - 指定蓝图内部静态文件夹
> - 指定蓝图内部模板目录

3.1 指定蓝图的url前缀

> 在应用中注册蓝图时使用 `url_prefix` 参数指定
>
> - `http://127.0.0.1:5000`
> - `http://127.0.0.1:5000/bp/blue`
>
> ```
> import json
> from flask import Flask, Blueprint
>
> bp=Blueprint('bp', __name__)
>
> @bp.route('/blue')
> def blue_view():
>  return 'this is blueprint'
>
> app = Flask(__name__)
> # 注册蓝图，url_prefix参数指定url前缀
> app.register_blueprint(bp, url_prefix='/bp')
>
> @app.route('/')
> def route_map():
>  """返回所有视图网址"""
>  rules_iterator = app.url_map.iter_rules()
>  return json.dumps({rule.endpoint: rule.rule for rule in rules_iterator})
>
> if __name__ == '__main__':
>  app.run(debug=True)
> ```

3.2 指定蓝图内部静态文件夹

3.2.1 注册蓝图的静态目录

> 和应用对象不同，蓝图对象创建时不会默认注册静态目录的路由。需要我们在 创建时指定 static_folder 参数。
>
> ```
> import json
> from flask import Flask, Blueprint
> # static_folder='bp_static' 指定蓝图内部静态文件夹
> bp=Blueprint('bp', __name__, static_folder='bp_static')
>
> @bp.route('/blue')
> def blue_view():
>  return 'this is blueprint'
>
> app = Flask(__name__)
> # 注册蓝图，url_prefix参数指定url前缀
> app.register_blueprint(bp, url_prefix='/bp')
>
> @app.route('/')
> def route_map():
>  """返回所有视图网址"""
>  rules_iterator = app.url_map.iter_rules()
>  return json.dumps({rule.endpoint: rule.rule for rule in rules_iterator})
>
> if __name__ == '__main__':
>  app.run(debug=True)
> ```
>
> 现在就可以使用 `/bp/bp_static/<filename>` 访问 `bp_static` 目录下的静态文件了。

3.2.2 设置蓝图静态文件的访问路径

> 也可通过 `static_url_path` 改变访问路径
>
> ```
> import json
> from flask import Flask, Blueprint
> # static_folder='bp_static' 指定蓝图内部静态文件夹
> # static_url_path='/lib' 改变访问路径为 /bp/lib/<filename>
> bp=Blueprint('bp', __name__, static_folder='bp_static', static_url_path='/lib')
>
> @bp.route('/blue')
> def blue_view():
>  return 'this is blueprint'
>
> app = Flask(__name__)
> # 注册蓝图，url_prefix参数指定url前缀
> app.register_blueprint(bp, url_prefix='/bp')
>
> ```

```
@app.route('/')
def route_map():
    """返回所有视图网址"""
    rules_iterator = app.url_map.iter_rules()
    return json.dumps({rule.endpoint: rule.rule for rule in rules_iterator})

if __name__ == '__main__':
    app.run(debug=True)
```

3.3 蓝图内部模板目录

> 蓝图对象默认的模板目录为系统的模版目录，可以在创建蓝图对象时使用 template_folder 关键字参数设置模板目录
>
> `http://127.0.0.1:5000/bp/blue`
>
> ```
> import json
> from flask import Flask, Blueprint
> # static_folder='bp_static' 指定蓝图内部静态文件夹
> # static_url_path='/lib' 改变访问路径为 /bp/lib/<filename>
> # template_folder='bp_templates' 改变蓝图的模板目录
> bp=Blueprint('bp', __name__,
>              static_folder='bp_static',
>              static_url_path='/lib',
>              template_folder='bp_templates')
>
> @bp.route('/blue')
> def blue_view():
>  return 'this is blueprint'
>
> app = Flask(__name__)
> # 注册蓝图，url_prefix参数指定url前缀
> app.register_blueprint(bp, url_prefix='/bp')
>
> @app.route('/')
> def route_map():
>  """返回所有视图网址"""
>  rules_iterator = app.url_map.iter_rules()
>  return json.dumps({rule.endpoint: rule.rule for rule in rules_iterator})
>
> if __name__ == '__main__':
>  app.run(debug=True)
> ```

3.4 拓展1:蓝图自定义名称的作用

> 蓝图自定义名称可以帮助我们做url_for跳转等工作

```python
from flask import Flask, Blueprint, render_template, url_for

bp=Blueprint('bp', __name__,
             static_folder='bp_static',
             static_url_path='/lib',
             template_folder='bp_templates')

@bp.route('/blue')
def blue_view():
    return render_template('3.html')

app = Flask(__name__)
app.register_blueprint(bp, url_prefix='/bp')

@app.route('/')
def index():
    # url_for函数，通过视图函数明获取它对应的url字符串
    # 某个视图url的字符串 = flask.url_for('视图函数名')
    # 蓝图的某个视图url的字符串 = flask.url_for('自定义蓝图名.视图函数名')
    return url_for('bp.blue_view') # '/bp/blue'

if __name__ == '__main__':
    app.run(debug=True)
```

3.5 拓展2:蓝图设置请求钩子,只会监听该蓝图注册的路由

> 关于 `请求钩子` ，我们会在后面学习

```python
from flask import Flask, Blueprint

bp=Blueprint('bp', __name__)

# 蓝图设置请求钩子，只会监听该蓝图注册的路由
# 在接收到请求之后，运行bp蓝图的视图函数之前运行
@bp.before_request
def before_request_func():
    print('只有访问蓝图的url才会看见我！')

@bp.route('/blue')
def blue_view():
    return 'blue'

app = Flask(__name__)
app.register_blueprint(bp, url_prefix='/bp')

@app.route('/')
def index():
```

```
    return 'index'

if __name__ == '__main__':
    app.run(debug=True)
```

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 18:04:09

请求与响应

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-05-08 00:12:06

处理请求

> 在视图编写中需要读取客户端请求携带的数据时，如何才能正确的取出数据呢？
>
> 请求携带的数据可能出现在HTTP报文中的不同位置，需要使用不同的方法来获取参数。
>
> - URL路径参数（动态路由）
> - URL路径参数的其他类型
> - 自定义URL路径参数的类型
> - 查询字符串
> - 请求体

[TOC]

- 1. URL路径参数(动态路由)
  - 1.1 基本用法
  - 1.2 URL路径参数的其他类型
  - 1.3 自定义URL路径参数的类型转换器
- 2. 查询字符串
- 3. 请求体
  - 3.1 POST `application/x www from urlencoded`
  - 3.2 POST `application/json`
  - 3.3 POST `multipart/form data`
  - 3.4 拓展:发送 `text/xml` 请求
- 4. 其他参数[查表]

1. URL路径参数(动态路由)

> 例如，有一个请求访问的接口地址为 `/users/123` ，其中 `123` 实际上为具体的请求参数，表明请求123号用户的信息。此时如何从url中提取出123的数据？
>
> Flask不同于Django直接在定义路由时编写正则表达式的方式，而是采用转换器语法：

1.1 基本用法

> ```
> from flask import Flask
> app = Flask(__name__)
>
> # 此处的<user_id>即是一个转换器，默认为字符串类型，即将该位置的数据以字符串格式进行匹配、并
> @app.route('/users/<user_id>')
> def get_user_id(user_id):
>  print(type(user_id))
>  return 'hello user {}'.format(user_id)
> ```

```
if __name__ == '__main__':
    app.run(debug=True)
```

1.2 URL路径参数的其他类型

- 在刚才的案例中， URL路径参数默认是str类型的，Flask也提供其他类型的转换器：

```
DEFAULT_CONVERTERS = {
 'default':          UnicodeConverter,
 'string':           UnicodeConverter,
 'any':              AnyConverter,
 'path':             PathConverter,
 'int':              IntegerConverter,
 'float':            FloatConverter,
 'uuid':             UUIDConverter,
}
```

- URL路径参数类型的转换器的具体使用

```
from flask import Flask
app = Flask(__name__)

# http://127.0.0.1:5000/users/123
@app.route('/users/<int:user_id>')
def user_info(user_id):
    print(type(user_id))
    return 'hello user: {}'.format(user_id)

# http://127.0.0.1:5000/path/a/b/c
@app.route('/path/<path:path_str>')
def path_info(path_str):
    print(type(path_str)) # str
    return 'hello path: {}'.format(path_str) # hello path: a/b/c

if __name__ == '__main__':
    app.run(debug=True)
```

1.3 自定义URL路径参数的类型转换器

如果遇到需要严格匹配提取 `/sms_codes/18512345678` 中的手机号数据，Flask内置的转换器就无法满足需求，此时需要自定义转换器。

```
from flask import Flask
from werkzeug.routing import BaseConverter
```

```
app = Flask(__name__)

# step 1. 创建转换器类，保存匹配时的正则表达式
# 注意regex名字固定
class MobileConverter(BaseConverter):
    """手机号格式"""
    regex = r'1[3-9]\d{9}'
# step 2. 将自定义转换器添加到转换器字典中，并指定转换器使用时名字为: mobile
app.url_map.converters['mobile'] = MobileConverter
# step 3. 在使用转换器的地方定义使用
@app.route('/mobile/<mobile:mob_num>')
def send_sms_code(mob_num):
    return 'mobile is: {}'.format(mob_num)

if __name__ == '__main__':
    app.run(debug=True)
```

- 测试：

  > `http://127.0.0.1:5000/mobile/13161933359` 正常返回
  >
  > `http://127.0.0.1:5000/mobile/1316193` 返回404
  >
  > - 思考：为什么第二个url返回404？
  > - 自定义的转换器类，本质是把url的格式固定死了。

2. 查询字符串

> `http://127.0.0.1:5000/s?a=1&b=2` ，url中 `?` 后边，以 `=` 连接键值对后，再以 `&` 连接的部分就是查询字符串部分。
>
> - 使用 `Flask.request.args` 来获取查询字符串
> - `Flask.request` 是flask封装提供的请求对象

```
from flask import Flask, request
app = Flask(__name__)

@app.route('/s')
def index():
    args_list = request.args.lists()
    for arg in args_list:
        print(arg)
    print(request.args.to_dict())
    print(request.args.get('a'))
    print(request.args.get('b', '没有b'))
    return request.args['a']
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

`http://127.0.0.1:5000/s?a=1&a=2` 试一试

3. 请求体

> POST请求存在四种请求体，前三种比较常见，他们都出现在请求头中的 `Content-Type` 中
>
> - application/x-www-from-urlencoded
> - application/json
> - multipart/form-data
> - text/xml

3.1 POST `application/x www from urlencoded`

> 使用 `Flask.request.form`
>
> ```
> from flask import Flask, request
> app = Flask(__name__)
>
> @app.route('/post_data', methods=['POST'])
> def post_data():
>  # application/x-www-form-urlencoded
>  print(request.form)
>  print(request.form['a'])
>  return request.form.get('b')
>
> if __name__ == '__main__':
>  app.run(debug=True)
> ```
>
> - 测试代码
>
>  > ```
>  > import requests
>  > # application/x-www-form-urlencoded格式
>  > data_dict = {"a": "1", "b": "2"}
>  > r = requests.post("http://127.0.0.1:5000/post_data", data=data_dict)
>  > print(r.text)
>  > print(r.status_code)
>  > ```

3.2 POST `application/json`

> 使用 `Flask.request.data` 或 `Flask.request.json`

```
from flask import Flask, request
app = Flask(__name__)

@app.route('/post_json', methods=['POST'])
def post_json():
    print(request.data)
    print(request.json)
    return request.data.decode()

if __name__ == '__main__':
    app.run(debug=True)
```

- 测试代码

```
import requests, json
headers = {'Content-Type': 'application/json'}
datas = json.dumps({"a": "1", "b": "2"})
r = requests.post("http://127.0.0.1:5000/post_json", data=datas, headers=he
print(r.text)
```

3.3 POST `multipart/form data`

我们来演示一下用 `multipart/form-data` 格式发送post请求上传文件

```
from flask import Flask, request
app = Flask(__name__)

@app.route('/upload', methods=['POST'])
def upload_file():
    f = request.files['zidingyi_name']
    f.save('./4_1_3_upload.py')
    # with open('./demo.png', 'wb') as new_file:
    #     new_file.write(f.read())
    return 'ok'

if __name__ == '__main__':
    app.run(debug=True)
```

- 测试代码

```
import requests
files = {"zidingyi_name": open("./4_1_3.py", "rb")}
r = requests.post("http://127.0.0.1:5000/upload", files=files)
print(r.text)
```

> - 思考：图片怎么上传？

3.4 拓展:发送 `text/xml` 请求

> Flask视图中可以使用request.data来获取text/xml形式的请求体，这里就不做演示了
>
> ```
> import requests
> headers = {"Content-Type": "text/xml"}
> datas = """<?xml version="1.0"?>
> <methodCall>
>  <methodName>examples.getStateName</methodName>
>  <params>
>  <param>
>  <value><i4>41</i4></value>
>  </param>
>  </params>
> </methodCall>"""
> r = requests.post("http://httpbin.org/post", data=datas, headers=headers)
> print(r.text)
> ```

- 拓展阅读:postman的使用

4. 其他参数[查表]

> `Flask.request` 还有其他参数，现将常用的列表如下：
>
> | 属性 | 说明 | 类型 |
> | --- | --- | --- |
> | data | 记录请求的数据，并转换为字符串 | * |
> | form | 记录请求中的表单数据 | MultiDict |
> | args | 记录请求中的查询参数 | MultiDict |
> | cookies | 记录请求中的cookie信息 | Dict |
> | headers | 记录请求中的报文头 | EnvironHeaders |
> | method | 记录请求使用的HTTP方法 | GET/POST |
> | url | 记录请求的URL地址 | string |
> | files | 记录请求上传的文件 | * |

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 18:15:01

处理响应

> - 如何在不同的场景里返回不同的响应信息？
    - 模板
    - 重定向
    - 返回JSON
    - 自定义状态码和响应头

[TOC]

- 1. 返回模板
- 2. 重定向
- 3. 返回JSON
- 4. 自定义状态码和响应头
    - 4.1 元祖方式
    - 4.2 make_response方式

1. 返回模板

> flask中集成了jinja2模板，在视图函数中，使用 `render_template` 方法渲染模板并返回
>
> - 例如，新建一个模板index.html
>
> ```
> <!DOCTYPE html>
> <html lang="en">
> <head>
>  <meta charset="UTF-8">
>  <title>Title</title>
> </head>
> <body>
> 我的模板html内容
> <br/>
> <br/>
> </body>
> </html>
> ```
>
> - 后端视图
>
> ```
> from flask import Flask, render_template
> app = Flask(__name__)
>
> @app.route('/')
> def index():
>  mstr = 'Hello 黑马程序员'
> ```

```
    mint = 10
    return render_template('index.html', my_str=mstr, my_int=mint)

if __name__ == '__main__':
    app.run()
```

2. 重定向

> 跳转其他url

```
from flask import Flask, redirect
app = Flask(__name__)

@app.route('/demo2')
def demo2():
    return redirect('http://www.itheima.com')

if __name__ == '__main__':
    app.run()
```

3. 返回JSON

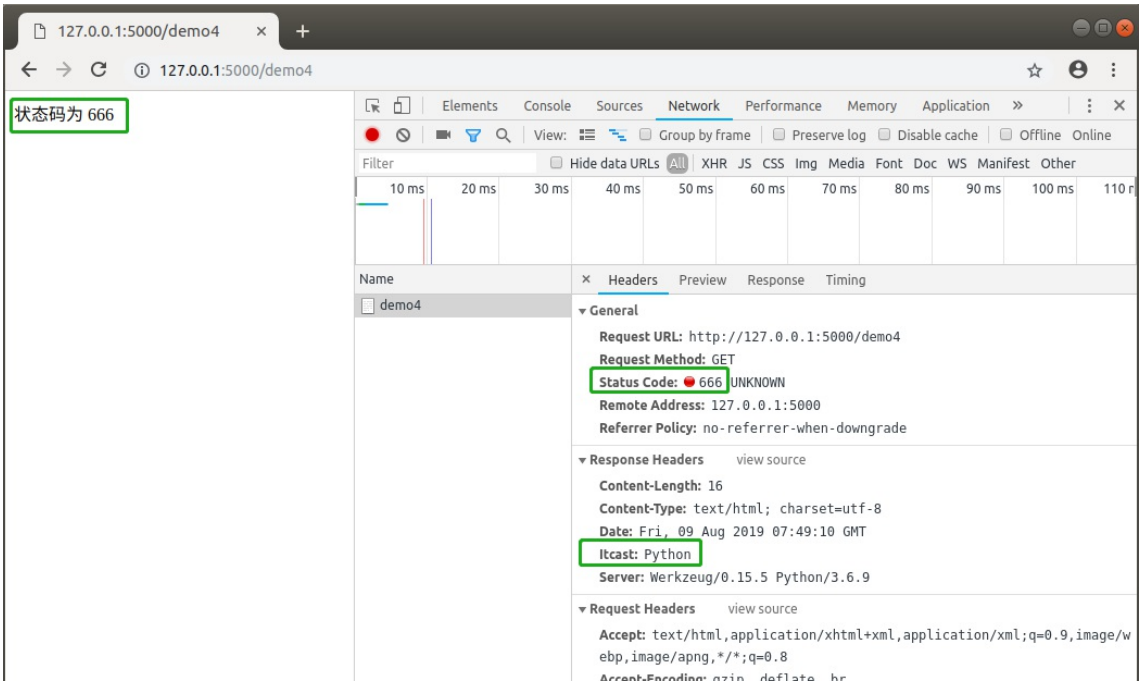
```
from flask import Flask, jsonify
app = Flask(__name__)

@app.route('/demo3')
def demo3():
    json_dict = {
        "user_id": 10,
        "user_name": "laowang"
    }
    return jsonify(json_dict)

if __name__ == '__main__':
    app.run()
```

4. 自定义状态码和响应头

4.1 元祖方式

> 视图函数可以返回一个元组，这样的元组必须是 **(response, status, headers)** 的形式，且至少包含一个元素。 status 值会覆盖状态代码， headers 可以是一个列表或字典，作为额外的消息标头值。

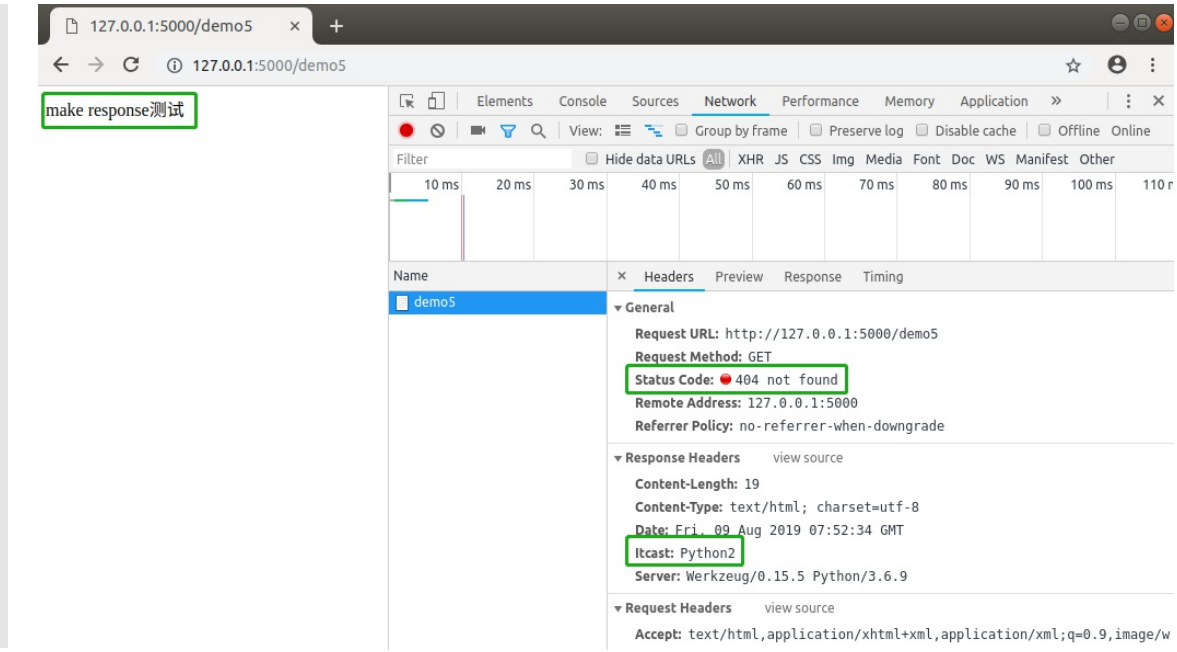
```
@app.route('/demo4')
def demo4():
    # return '状态码为 666', 666
    # return '状态码为 666', 666, [('Itcast', 'Python')]
    return '状态码为 666', 666, {'Itcast': 'Python'}
```



4.2 make_response方式

flask.make_response函数构造response响应对象

```
@app.route('/demo5')
def demo5():
    resp = make_response('make response测试')
        resp.headers[“Itcast”] = “Python2”
        resp.status = “404 not found”
    return resp
```



toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 18:16:56

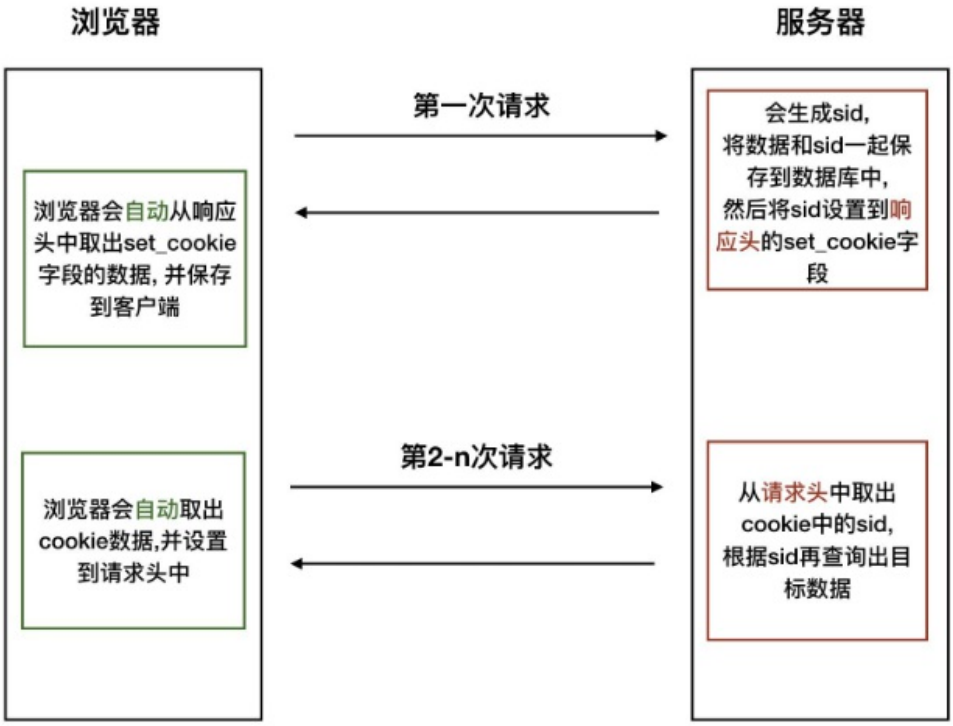
Cookie与Session

[TOC]

- 1. 关于cookie与session
  - 1.1 cookie和session在请求过程中的交互过程
  - 1.2 cookie
  - 1.3 session
- 2. 在flask中操作Cookie
  - 2.1 运行下边的代码
  - 2.2 用浏览器访问查看效果
    - 2.2.1 `/set_cookie`
    - 2.2.2 `/get_cookie`
    - 2.2.3 `/delete_cooke`
- 3. 在flask中操作Session
  - 3.1 运行下边的代码
  - 3.2 打开浏览器测试
    - 3.2.1 `/set_session`
    - 3.2.2 `/get_session`
    - 3.2.3 `/delete_session`
    - 3.2.4 `/check_session`
- 4. 思考:在flask中,cookie和session区别和联系?

1. 关于cookie与session

1.1 cookie和session在请求过程中的交互过程



sessionid	mobile	name
1	134xxx	zs
2	152xx	lisi

1.2 cookie

> - 一旦保存了cookie数据，访问该网站的任意路由时，cookie数据都会放在请求头中被发送
> - cookie数据智慧发送给生成该cookie网站（同源策略）
> - cookie在前端可以通过js操作，浏览器本身能够执行js

1.3 session

> - session依赖cookie机制
> - 传统的session机制和flask session机制区别
> - 传统session把信息存储在服务器
> - flask session把信息存储在客户端
> - flask-session模块可以实现传统session机制

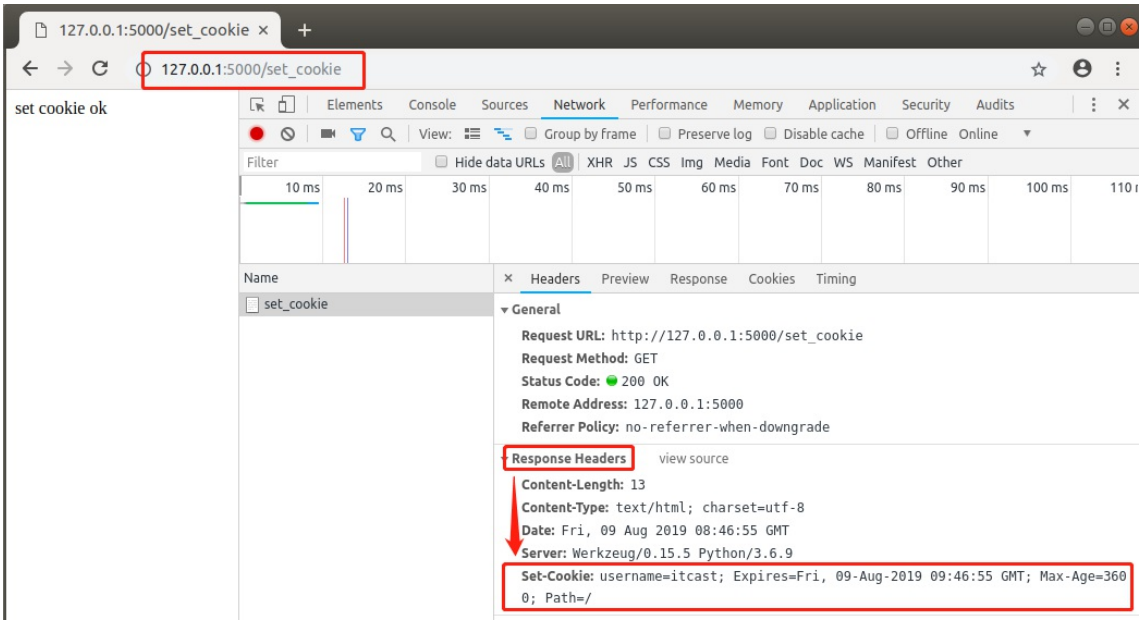
2. 在flask中操作Cookie

2.1 运行下边的代码

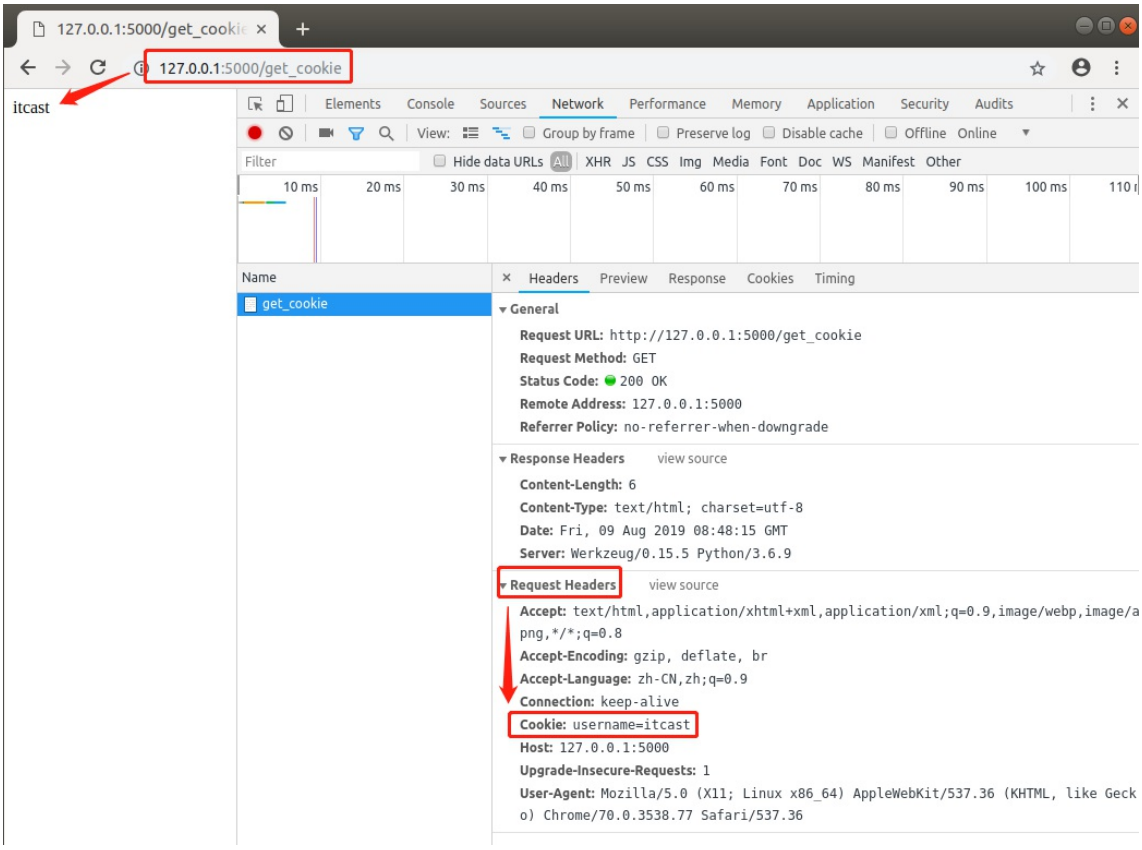
```python
from flask import Flask, make_response, request

app = Flask(__name__)


@app.route('/set_cookie')
def set_cookie():
    resp = make_response('set cookie ok')
    # 设置cookie:  key, value, max_age(过期时间, 秒, 默认浏览器关闭过期, 但现在很多浏览器
    resp.set_cookie('username', 'itcast', max_age=3600)
    return resp

@app.route('/get_cookie')
def get_cookie():
    # 读取请求中cookie的特定key的值
    resp_str = request.cookies.get('username')
    return resp_str

@app.route('/delete_cookie')
def delete_cookie():
    response = make_response('cookie is deleted.')
    # 删除cookie中的某个key
    response.delete_cookie('username')
    return response

if __name__ == '__main__':
    app.run(debug=True)
```

2.2 用浏览器访问查看效果

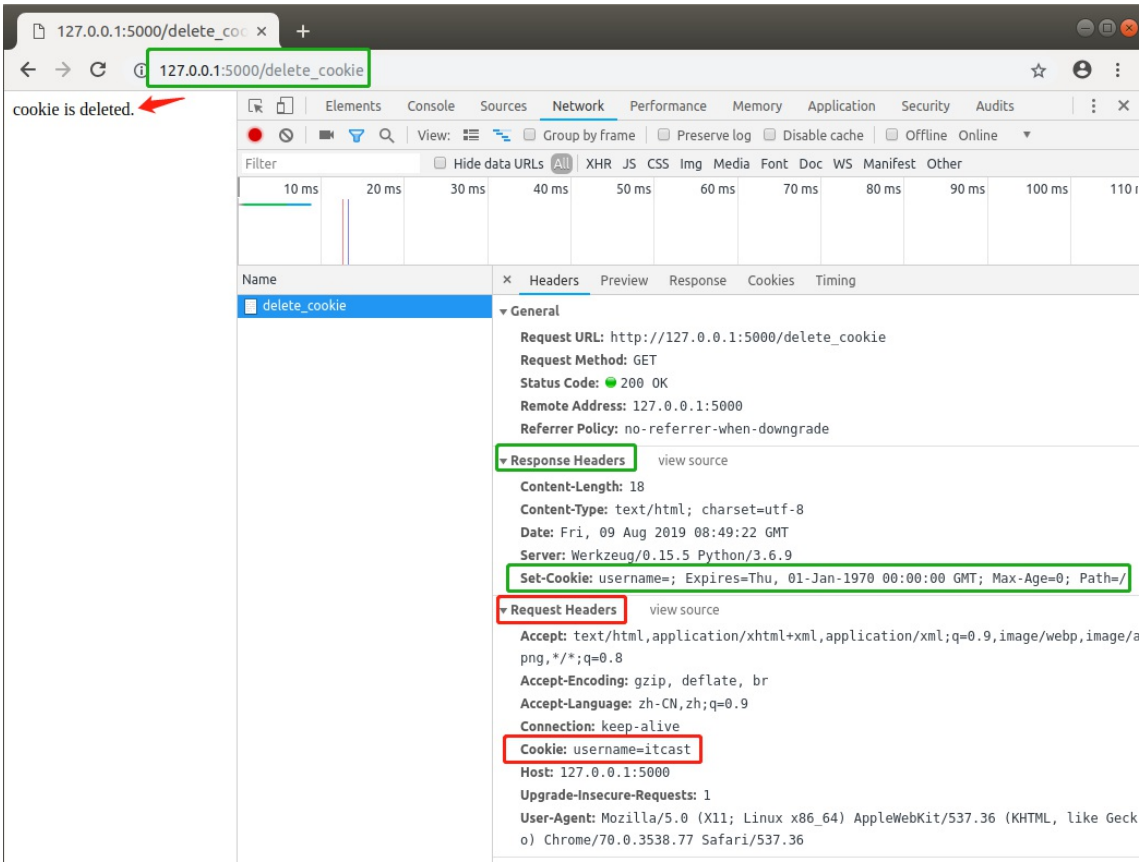
2.2.1 `/set_cookie`



2.2.2 `/get_cookie`



2.2.3 `/delete_cooke`



3. 在flask中操作Session

3.1 运行下边的代码

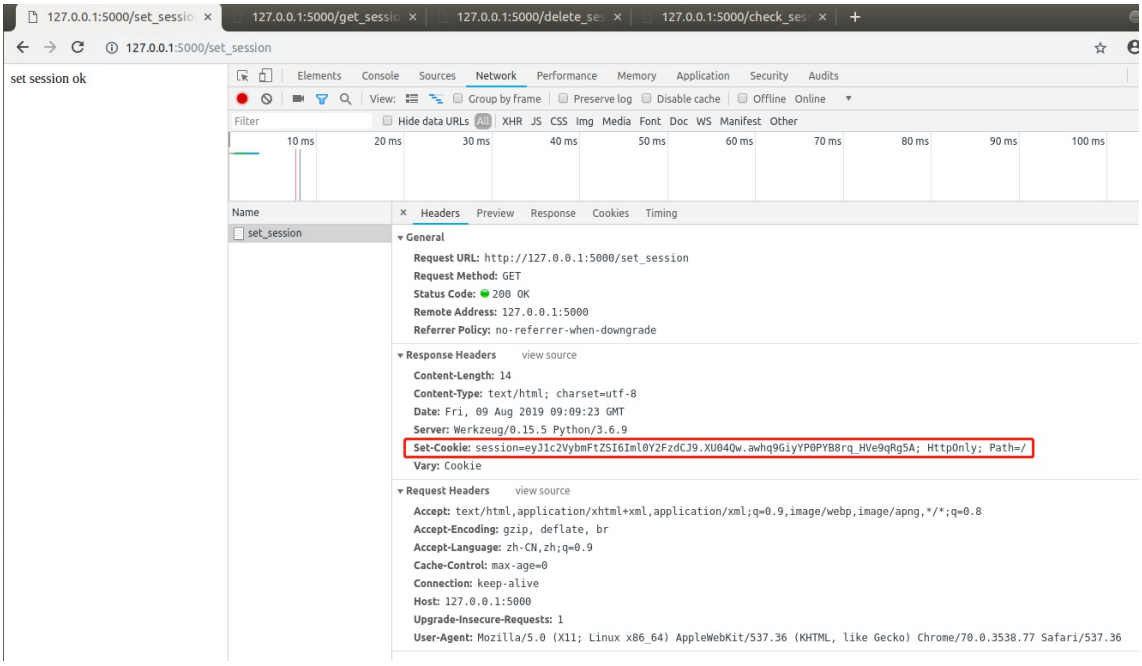
> 在flask中需要先设置SECRET_KEY

```
from datetime import timedelta
from flask import Flask, session

app = Flask(__name__)

# 在flask中需要先设置SECRET_KEY，才可以使用session
class DefaultConfig(object):
    SECRET_KEY = 'fih9fh9eh9gh2'
app.config.from_object(DefaultConfig)

@app.route('/set_session')
def set_session():
    session['username'] = 'itcast' # 设置session的key value
    # session.permanent = True # 设置session永久有效
    app.permanent_session_lifetime = timedelta(seconds=100) # 设置session过期时间
    # session的默认过期时间是31天
    return 'set session ok'
```

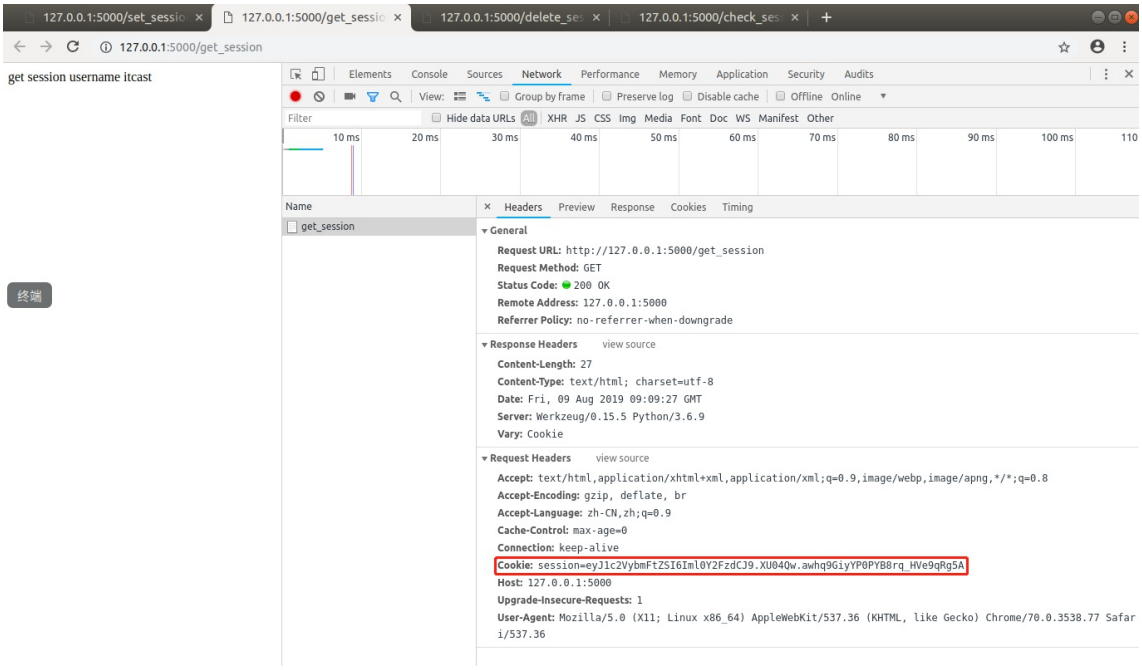
```python
@app.route('/get_session')
def get_session():
    username = session.get('username') # 获取session中某个key的值
    return 'get session username {}'.format(username)

@app.route('/delete_session')
def delete_session():
    _ = session.pop('username', None) # 弹出指定key的值，如果取不出，则返回设置的默认值
    session.clear() # 删除全部session信息
    return 'session is deleted.'

@app.route('/check_session')
def check_session():
    return session.pop('username', 'session is deleted.')

if __name__ == '__main__':
    app.run(debug=True)
```

3.2 打开浏览器测试

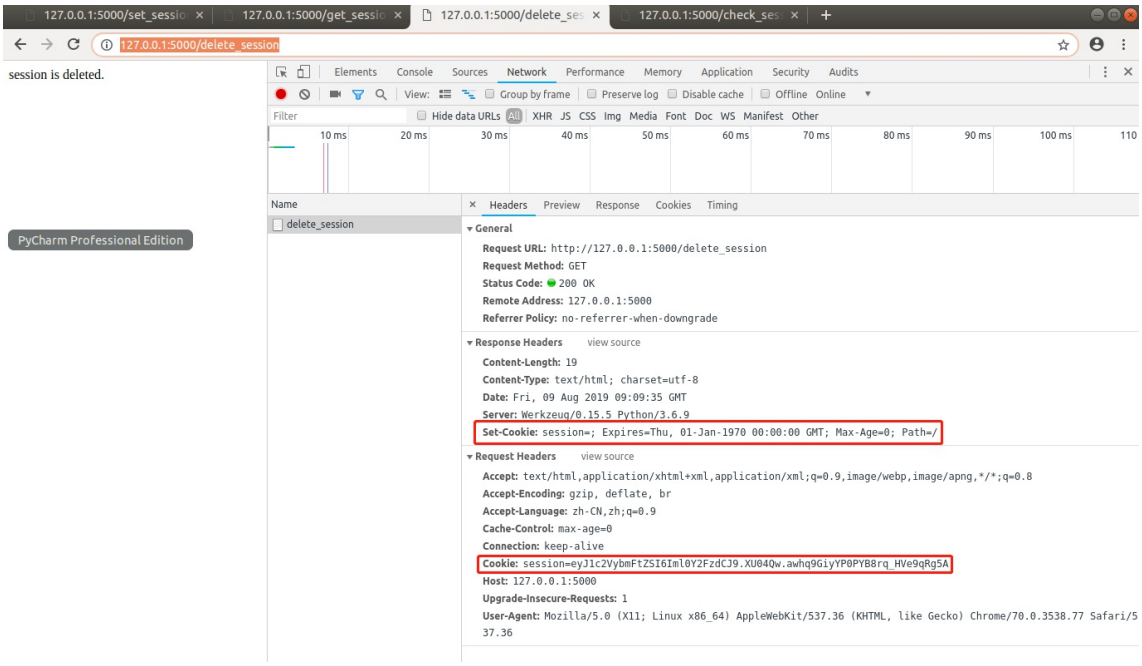
3.2.1 `/set_session`



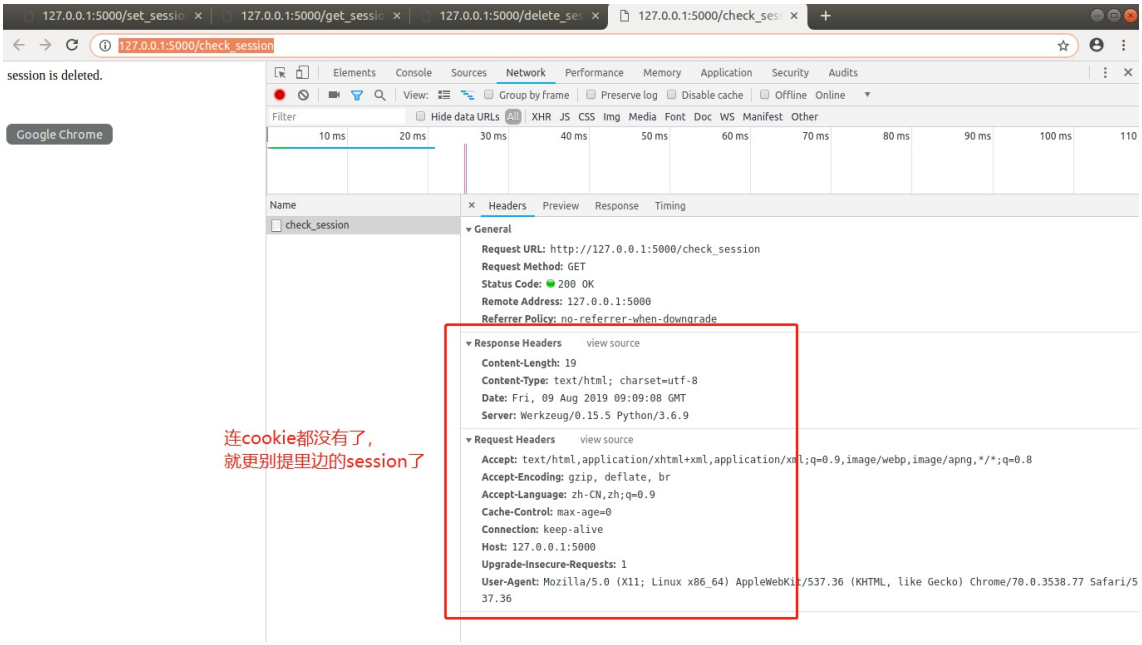
3.2.2 `/get_session`



3.2.3 `/delete_session`



3.2.4 `/check_session`



4. 思考:在flask中,cookie和session区别和联系?

> - 存储的位置都是在客户端浏览器
> - session基于客户端的cookie来存储
> - session基于flask框架自身的功能自行对信息进行加密

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 18:22:27

请求钩子与上下文

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-05-08 01:30:20

异常处理

[TOC]

- 1. HTTP异常主动抛出
- 2. 捕获错误
- 3. 完整示例代码解读

1. HTTP异常主动抛出

> - abort 方法
> - 抛出一个给定状态代码的 HTTPException 或者 指定响应，例如想要用一个页面未找到异常来终止请求，你可以调用 abort(404)。
> - 参数：
> - code – HTTP的错误状态码
>
> ```
> # abort(404)
> abort(500)
> ```
>
> 抛出状态码的话，只能抛出 HTTP 协议的错误状态码

2. 捕获错误

> - errorhandler 装饰器
> - 注册一个错误处理程序，当程序抛出指定错误状态码的时候，就会调用该装饰器所装饰的方法
> - 参数：
> - code_or_exception – HTTP的错误状态码或指定异常
> - 例如统一处理状态码为500的错误给用户友好的提示：
>
> ```
> @app.errorhandler(500)
> def internal_server_error(e):
>  return '服务器搬家了'
> ```
>
> - 捕获指定异常
>
> ```
> @app.errorhandler(ZeroDivisionError)
> def zero_division_error(e):
>  return '除数不能为0'
> ```

3. 完整示例代码解读

```
from flask import Flask, abort

app = Flask(__name__)

# app.errorhandler 可以捕获http错误 和 系统内置错误
@app.errorhandler(404) # 捕获http-404错误
def eror_404(e): # 必须接收异常信息参数
    return '走丢了！ 404 ' # 一旦触发捕获的异常给前端返回的内容

@app.route('/404')
def index1():
    abort(404) # 主动抛出http异常，被@app.errorhandler(404)捕获了异常
    return 'heihei'

@app.route('/500')
def index2():
    abort(500) # 主动抛出http异常，
    return

@app.errorhandler(ZeroDivisionError) # 捕获系统内置错误
def error_zero(e):
    return '除数真的不能为0'

@app.route('/0')
def index3():
    1 / 0 # 强行造的异常
    return '除数不能为0'

if __name__ == '__main__':
    app.run(debug=True)
```

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 18:24:23

请求钩子

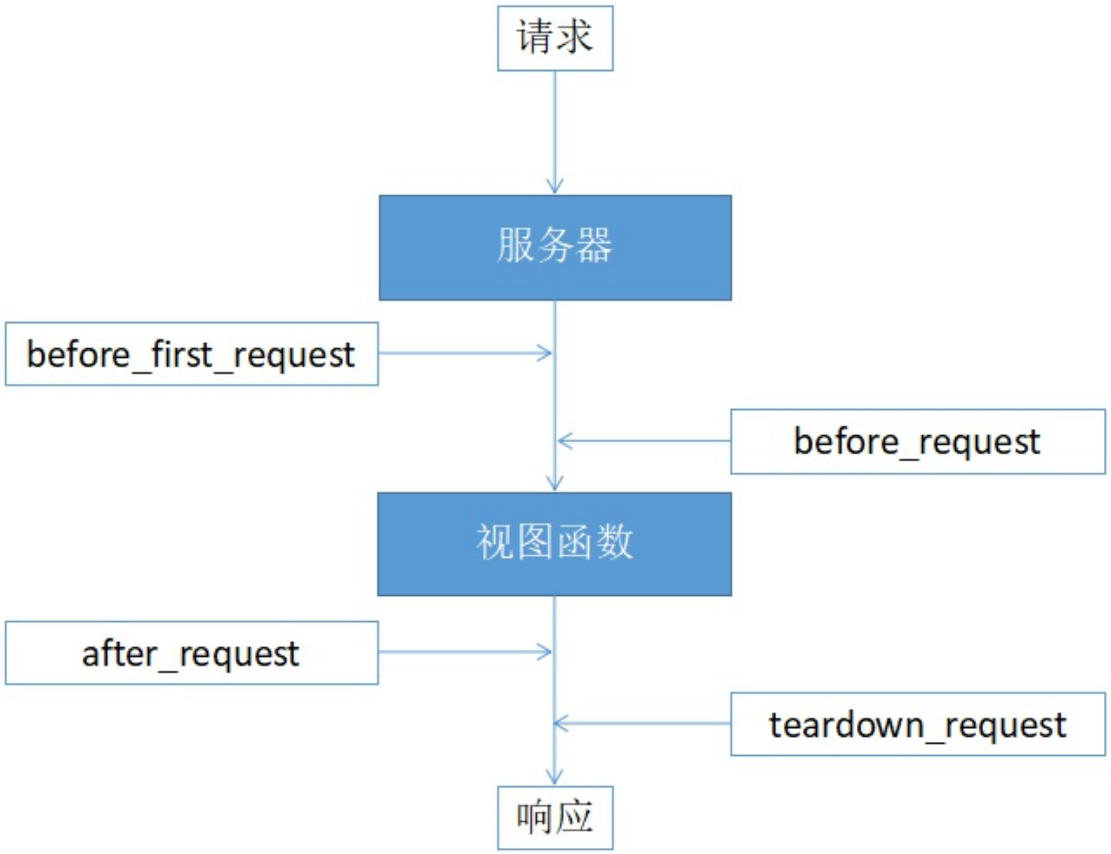
> 在客户端和服务器交互的过程中，有些准备工作或扫尾工作需要处理，比如：
>
> - 在请求开始时，建立数据库连接；
> - 在请求开始时，根据需求进行权限校验；
> - 在请求结束时，指定数据的交互格式；
>
> 为了让每个视图函数避免编写重复功能的代码，Flask提供了通用设施的功能，即请求钩子。

[TOC]

- 1. 认识请求钩子
- 2. 代码实现

1. 认识请求钩子

> 请求钩子是通过装饰器的形式实现，支持如下四种请求钩子：
>
> - before_first_request
> - 整个进程在处理第一个请求前执行
> - before_request
> - 在每次请求前执行
> - 如果在执行before_request时返回了一个响应，视图函数将不再被调用
> - after_request
> - 如果没有抛出错误，在每次请求后执行
> - 接受一个固定参数：视图函数返回的响应对象
> - 在此函数中可以对响应值在返回之前做最后一步修改处理
> - 在after_request处理的最后，必须将传入的响应对象返回
> - teardown_request：
> - 在每次请求后执行
> - 接受一个参数：错误信息，如果有相关错误抛出



2. 代码实现

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def index():
    print('*** [执行了视图]')
    a = 1 / 0
    return 'index'

# 在第一次请求之前调用，可以在此方法内部做一些初始化操作
@app.before_first_request
def before_first_request():
    print('*** [before_first_request] flask app进程开启后，收到第一个请求时执行，此时请求

# 另一种设置的语法
def before_request():
    print('*** [before_request] 每次“请求之前”调用，一般完成一些请求准备工作，如参数校验，
app.before_request(before_request) # 通过app对象直接注册钩子函数
```

```
# 在执行完视图函数之后会调用，并且会把视图函数所生成的响应传入,可以在此方法中对响应做最后一步
@app.after_request
def after_request(response): # 必须定义形参来接受响应对象
    print('*** [after_request] 在执行完视图函数之后会调用，并且会把视图函数所生成的响应传
    return response # 对响应内容加工后，必须返回相应对象

"""teardown_request必须关闭调试模式才能生效！"""
# 请每一次请求之后都会调用，无论是否出现异常都会执行，必须接受参数是服务器出现的错误信息
@app.teardown_request
def teardown_request(e): # 如果没有错误信息，  e=None
    ret = '*** [teardown_request] 请每一次请求之后都会调用，无论是否出现异常都会执行，必须
    print(ret)
    return ret

if __name__ == '__main__':
    app.run(debug=False)
```

- 输出打印：

```
*** [before_first_request] flask app进程开启后，收到第一个请求时执行，此时请求还没有进入视
*** [before_request] 每次“请求之前”调用，一般完成一些请求准备工作，如参数校验，黑名单过滤，
*** [执行了视图]
*** [after_request] 在执行完视图函数之后会调用，并且会把视图函数所生成的响应传入,可以在此方
*** [teardown_request] 请每一次请求之后都会调用，无论是否出现异常都会执行，必须接受参数是服
```

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 18:26:42

上下文

> 上下文：即语境，语意，在程序中可以理解为在代码执行到某一时刻时，根据之前代码所做的操作以及下文即将要执行的逻辑，可以决定在当前时刻下可以使用到的变量，或者可以完成的事情。
>
> - Flask中有两种上下文，请求上下文和应用上下文：
> - 请求上下文
> - request
> - session
> - request_context
> - 应用上下文
> - current_app
> - g对象
> - app_context
> - Flask中上下文对象：相当于一个容器，保存了 Flask 程序运行过程中的一些信息。

[TOC]

- 1. 请求上下文(request context)
- 2. 应用上下文(application context)
  - 2.1 current_app
  - 2.2 g对象
  - 2.3 拓展: `app_context`
- 3. 案例
  - 3.1 需求:写一个flask项目符合下列要求
  - 3.2 代码实现

1. 请求上下文(request context)

> 思考：在视图函数中，如何取到当前请求的相关数据？比如：请求地址，请求方式，cookie等等
>
> 在 flask 中，可以直接在视图函数中使用 **request** 这个对象进行获取相关数据，而 **request** 就是请求上下文的请求对象，保存了当前本次请求的相关数据；另外还有**session**
>
> - request
> - 封装了HTTP请求的内容，针对的是http请求。举例：user = request.args.get('user')，获取的是get请求的参数。
> - 每一次请求周期结束后，都会清空该次请求的相关信息
> - session
> - 用来记录请求会话中的信息，针对的是用户信息。举例：session['name'] =

user.id，可以记录用户信息。还可以通过session.get('name')获取用户信息。
    - 每一次请求周期结束后，都会清空该次请求的相关信息
- request_context【拓展】

> `request_context` 为我们提供了请求上下文环境，允许我们在外部使用请求上下文 `request` 、 `session`
>
> 可以通过with语句进行使用
>
> ```
> # 注意：在python终端/python console中进行下面的代码
> from flask import Flask, request
> app = Flask('')
> request.args
> # 此时输出一坨异常信息，因为没有上下文环境
> environ = {'wsgi.version':(1,0), 'wsgi.input': '', 'REQUEST_METHOD': 'GET',
> # environ模拟了解析客户端请求之后的wsgi字典数据
> with app.request_context(environ):  # 借助with语句使用request_context创建请求
>  print(request.path)
> # 成功输出：/hahahhaha
> ```

2. 应用上下文(application context)

> 它的字面意思是 应用上下文，但它不是一直存在的，它只是request context 中的一个对 app 的代理(人)，所谓local proxy。它的作用主要是帮助 request 获取当前的应用，它是伴 request 而生，随 request 而灭的。
>
> - 应用上下文对象有：
> - current_app
> - g
> - app_context

2.1 current_app

> - 应用程序上下文,用于存储应用程序中的变量，可以通过current_app.name打印当前app的名称，也可以在current_app中存储一些变量，例如：
>
> > - 应用的启动脚本是哪个文件，启动时指定了哪些参数
> > - 加载了哪些配置文件，导入了哪些配置
> > - 连了哪个数据库
> > - 有哪些public的工具类、常量
> > - 应用跑在哪个机器上，IP多少，内存多大
>
> - 示例
>
>  ```
>  from flask import Flask, current_app
>  ```

```python
app = Flask(__name__)

# 为了方便在各个视图中使用，可以将一些自定义内容保存到flask app中
# 后续可以在视图中使用current_app.xxx获取
app.xxx = '这个app.xxx中的xxx是我自己起的名字'

@app.route('/1')
def route1():
    return current_app.xxx

@app.route('/2')
def route2():
    return current_app.xxx

if __name__ == '__main__':
    app.run(debug=True)
```

- 结论：
  - `current_app` 就是当前运行的**flask app**，在代码不方便直接操作**flask**的**app**对象时，可以操作 `current_app` 就等价于操作**flask app**对象

2.2 g对象

g 作为 flask 程序全局的一个临时变量，充当中间媒介的作用，我们可以通过它在一次请求调用的多个函数间传递一些数据。每次请求都会重设这个变量。

```python
from flask import Flask, request, g

app = Flask(__name__)

def db_query():
    print('user_id={} user_name={}'.format(g.user_id, g.user_name))

@app.route('/')
def index():
    g.user_id = request.args['user_id']
    g.user_name = request.args['name']
    db_query()
    return 'user_id={} user_name={}'.format(g.user_id, g.user_name)

if __name__ == '__main__':
    app.run(debug=True)
    # http://127.0.0.1:5000/?user_id=1&name=白色哈士奇
    # http://127.0.0.1:5000/?user_id=2&name=黑色哈士奇
```

> - 重要结论：**g**对象和**request**对象一样：生命周期只在那一次完整的请求中

2.3 拓展：`app_context`

> `app_context` 为我们提供了应用上下文环境，允许我们在外部使用应用上下文 `current_app` 、 `g` 等
>
> 和 `request_context` 一样，可以通过 `with` 语句进行使用
>
> ```
> # 注意：在python终端/python console中进行下面的代码
> from flask import Flask, current_app
> app = Flask('')
> app.zuozhe = '无敌哈士奇'
> current_app.zuozhe
> # 一大坨报错信息，原因：没有上下文环境
> # 借助with语句使用app_context创建应用上下文
> with app.app_context():
>  print(current_app.zuozhe)
> # 成功输出：'无敌哈士奇'
> ```

3. 案例

> 模拟用户访问个人中心涉及的权证认证场景

3.1 需求:写一个flask项目符合下列要求

> - 技术要求使用 `session` 、 `g对象` 、 `请求钩子`
> - 接口要求
> - `/`
> - 如果未登录，返回 `首页`
> - 如果已登录，返回 `欢迎回来，xxx`
> - `/user`
> - 如果未登录，返回 `无法访问个人中心，请登录`
> - 如果已登录，返回 `个人中心`
> - `/login`
> - 模拟登录，用户查询字符串传递用户昵称

3.2 代码实现

> ```
> from flask import Flask, session, g, abort
>
> app = Flask(__name__)
> app.secret_key = 'fdsfsdf'
> ```

```python
@app.errorhandler(401)
def error_handler(e):
    return '无法访问个人中心，请登录'

# 使用请求钩子，把用户信息保存在g变量中
@app.before_request
def get_user_info():
    """获取用户信息，其实就是获取认证信息"""
    g.name = session.get('name', None)

# 需求：对指定的/user视图进行访问限制 如个人中心必须登录后才能访问
# 解决办法：    方案1：把需要限制访问的视图放入蓝图中，对蓝图设置请求钩子
# 这里采用：    方案2：实现装饰器
def login_required(func):
    def wrapper(*args, **kwargs):
        if g.name: # 已登录，允许执行视图函数
            return func(*args, **kwargs)
        else:
            # 400 语法/参数错误 401 未认证 403 已认证但权限不够
            # 404 资源不存在 405 请求方式不支持
            abort(401)
    return wrapper

@app.route('/user')
@login_required
def user(): # 需要已登录状态才能访问
    return '个人中心'

@app.route('/')
def index():
    if g.name: # 已经登录
        return '欢迎回来，%s' % g.name
    else: # 未登录
        return '首页'

@app.route('/login')
def login():
    # 模拟用户
    session['name'] = '无敌哈士奇' # 登录成功，记录认证信息
    return '用户登录'

if __name__ == '__main__':
    app.run(debug=True)
```

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 18:30:49

Flask-RESTful

> 课后阅读：理解restful

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-09 20:10:45

flask-restful的helloworld

> **Flask-RESTful**是用于快速构建**REST API**的**Flask**扩展模块。

[TOC]

- 1. 安装
- 2. Hello World

1. 安装

```
pip install flask-restful
```

2. Hello World

```
from flask import Flask
from flask_restful import Resource, Api

app = Flask(__name__)
# 1. 创建api对象，加载app对象
api = Api(app)

# 2. 定义视图类 继承Resource
class HelloWorldResource(Resource):
    # 类视图的响应content-type会自动设为application/json
    # 类视图方法直接返回字典，能自动序列化为json返回给客户端
    # 类视图的方法名对应请求方式
    def get(self):
        return {'hello': 'world'}

    def post(self):
        return {'msg': 'post hello world'}

# 3. 添加类视图
api.add_resource(HelloWorldResource, '/')

if __name__ == '__main__':
    app.run(debug=True)
```

- 测试代码：

```
import requests
```

```
url = 'http://127.0.0.1:5000/'

resp = requests.get(url)
print(resp.json())

resp = requests.post(url)
print(resp.text)
```

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 18:32:29

关于视图

[TOC]

- 1. 为flask restful路由起名
- 2. flask restful使用蓝图
- 3. flask restful视图使用装饰器

1. 为flask restful路由起名

> 通过endpoint参数为路由起名，默认名称是视图类名的小写
>
> ```
> api.add_resource(HelloWorldResource, '/', endpoint='HelloWorld')
> ```
>
> 验证：
>
> ```
> print(app.url_map)
> ```

2. flask restful使用蓝图

> ```
> from flask import Flask, Blueprint
> from flask_restful import Api, Resource
>
> # 视图类
> class UserProfileResource(Resource):
>  def get(self):
>  return {'msg': 'get user profile'}
>
> app = Flask(__name__)
> # 1. 定义蓝图
> user_bp = Blueprint('user', __name__)
> # 2. 在restful API中使用蓝图
> user_api = Api(user_bp)
> # 3. 注册API路由
> user_api.add_resource(UserProfileResource, '/users/profile', endpoint='hahaha')
> # 4. app注册蓝图
> app.register_blueprint(user_bp)
>
> if __name__ == '__main__':
>  app.run(debug=True)
> ```

3. flask restful视图使用装饰器

> - 使用视图类属性 `method_decorators` 添加装饰器
> > - 接收列表：
> > - 视图类中所有方法都被装饰
> > - 可以有多个装饰器
> > - 接收字典：
> > - 视图类中规定的方法被装饰
> > - 可以有多个装饰器

```
from flask import Flask, Blueprint
from flask_restful import Api, Resource


def decorator1(func):
    def wrapper(*args, **kwargs):
        print('decorator1')
        return func(*args, **kwargs)
    return wrapper

def decorator2(func):
    def wrapper(*args, **kwargs):
        print('decorator2')
        return func(*args, **kwargs)
    return wrapper

# 视图类
class UserProfileResource(Resource):
    # 为类视图中的所有方法添加装饰器
    # method_decorators = [decorator1, decorator2]
    # 为类视图中不同的方法添加不同的装饰器
    method_decorators = {'get': [decorator1, decorator2],
                         'post': [decorator1]}
    def get(self):
        return {'msg': 'get'}
    def post(self):
        return {'msg': 'post'}

app = Flask(__name__)
# 1. 定义蓝图
user_bp = Blueprint('user', __name__)
# 2. 在restful API中使用蓝图
user_api = Api(user_bp)
# 3. 注册API路由
user_api.add_resource(UserProfileResource, '/users/profile', endpoint='hahaha')
# 4. app注册蓝图
```

```
app.register_blueprint(user_bp)

if __name__ == '__main__':
    app.run(debug=True)
```

- 测试代码

```
import requests

url = 'http://127.0.0.1:5000/users/profile'

resp = requests.get(url)
print(resp.json())

resp = requests.post(url)
print(resp.text)
```

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 18:34:30

关于请求处理

> 对于请求的处理，其实主要是对请求数据的解析
>
> Flask-RESTful 提供了 `RequestParser` 类，用来帮助我们检验和转换请求数据。

[TOC]

- 1. 使用步骤
- 2. 请求数据检验转换方法的参数说明
  - 2.1 代码实现
  - 2.2 参数总结[查表]
    - 2.2.1 required
    - 2.2.2 help
    - 2.2.3 action
    - 2.2.4 type
    - 2.2.5 location

1. 使用步骤

> - 步骤：
>
> > 1. 创建 `RequestParser` 对象
> > 2. 向 `RequestParser` 对象中添加需要检验或转换的参数声明
> > 3. 使用 `parse_args()` 方法启动检验处理
> > 4. 检验之后从检验结果中获取参数时可按照字典操作或对象属性操作
>
> - 伪代码如下：
>
>  > ```
>  > # 导包
>  > from flask_restful import reqparse
>  > # 1. 创建RequestParser对象
>  > parser = reqparse.RequestParser()
>  > # 2. 向RequestParser对象中添加需要检验或转换的参数声明
>  > parser.add_argument('rate', type=int, help='Rate cannot be converted', loca
>  > # 添加另外一个参数声明
>  > parser.add_argument('name')
>  > # 3. 使用parse_args()方法启动检验处理
>  > args = parser.parse_args()
>  > # 4. 检验之后从检验结果中获取参数时可按照字典操作或对象属性操作
>  > args.rate
>  > # 或 args['rate']
>  > ```

2. 请求数据检验转换方法的参数说明

2.1 代码实现

```
import re
from flask import Flask, Blueprint
from flask_restful import Api, Resource
from flask_restful import inputs
from flask_restful.reqparse import RequestParser

# 视图类
class UserProfileResource(Resource):
    def get(self):
        # step 1. 创建`RequestParser`对象
        rp = RequestParser()
        # step 2. 向`RequestParser`对象中添加需要检验或转换的参数声明
        parser = rp.add_argument('name', # 参数名
                                 required=False, # 描述请求是否一定要携带对应参数
                                                 # 默认为False: 在客户端请求未携带参数时
                                                 # True: 若未携带，则校验失败，向客户端
                                 help='参数检验错误时返回的错误描述信息',
                                 action='append', # 以列表追加保存所有同名参数的值
                                 # action='store', # 对于请求参数中出现多个同名参数时，
                                 type=str, # 参数类型检验
                                 # type=mobile, # 自定义类型检验，mobile是自定义的类型检
                                 # type=inputs.regex(r'^\d{2}&'), # flask_restful.inp
                                 # location='args', # 描述参数应该在请求数据中出现的位置
                                 location = ['headers', # 也可指明多个位置
                                             'cookies',
                                             'json',
                                             'form',
                                             'args',
                                             'files'])
        # setp 3. 使用`parse_args()`方法启动检验处理
        args = parser.parse_args()
        # setp 4. 检验之后从检验结果中获取参数时可按照字典操作或对象属性操作
        print(args['name'])
        return {'msg': args['name']}

def mobile(mobile_str):
    """flask_restful.reqparse.RequestParser.type参数自定义类型检验函数
    检验手机号格式
    :param mobile_str: 被检验字符串 str
    :return: mobile_str str
    """
    if re.match(r'^1[3-9]\d{9}$', mobile_str):
        return mobile_str
```

```
    else:
        raise ValueError('{} is not a valid mobile'.format(mobile_str))

app = Flask(__name__)
# 1. 定义蓝图
user_bp = Blueprint('user', __name__)
# 2. 在restful API中使用蓝图
user_api = Api(user_bp)
# 3. 注册API路由
user_api.add_resource(UserProfileResource, '/', endpoint='hahaha')
# 4. app注册蓝图
app.register_blueprint(user_bp)

if __name__ == '__main__':
    app.run(debug=True)
```

2.2 参数总结[查表]

2.2.1 required

> 描述请求是否一定要携带对应参数，默认值为**False**
>
> - True 强制要求携带
> - 若未携带，则校验失败，向客户端返回错误信息，状态码400
> - False 不强制要求携带
> - 若不强制携带，在客户端请求未携带参数时，取出值为None
>
> ```
> class DemoResource(Resource):
>  def get(self):
>  rp = RequestParser()
>  rp.add_argument('a', required=False)
>  args = rp.parse_args()
>  return {'msg': 'data={}'.format(args.a)}
> ```

2.2.2 help

> 参数检验错误时返回的错误描述信息
>
> ```
> rp.add_argument('a', required=True, help='missing a param')
> ```

2.2.3 action

> 描述对于请求参数中出现多个同名参数时的处理方式
>
> - action='store'` 保留出现的第一个， 默认

- `action='append'` 以列表追加保存所有同名参数的值

```
rp.add_argument('a', required=True, help='missing a param', action='append')
```

2.2.4 type

描述参数应该匹配的类型，可以使用python的标准数据类型string、int，也可使用Flask-RESTful提供的检验方法，还可以自己定义

- 标准类型

```
rp.add_argument('a', type=int, required=True, help='missing a param', action='ap
```

- 拓展一：`flask_restful.inputs` 提供的类型
  - `url`
  - `regex(指定正则表达式)`

    ```
    from flask_restful import inputs
    rp.add_argument('a', type=inputs.regex(r'^\d{2}&'))
    ```

  - `natural` 自然数0、1、2、3...
  - `positive` 正整数 1、2、3...
  - `int_range(low ,high)` 整数范围

    ```
    rp.add_argument('a', type=inputs.int_range(1, 10))
    ```

  - boolean
- 拓展二：自定义类型

```
def mobile(mobile_str):
    """
    检验手机号格式
    :param mobile_str: str 被检验字符串
    :return: mobile_str
    """
    if re.match(r'^1[3-9]\d{9}$', mobile_str):
        return mobile_str
    else:
        raise ValueError('{} is not a valid mobile'.format(mobile_str))

rp.add_argument('a', type=mobile)
```

2.2.5 location

- 描述参数应该在请求数据中出现的位置

```
# Look only in the POST body
parser.add_argument('name', type=int, location='form')

# Look only in the querystring
parser.add_argument('PageSize', type=int, location='args')

# From the request headers
parser.add_argument('User-Agent', location='headers')

# From http cookies
parser.add_argument('session_id', location='cookies')

# From json
parser.add_argument('user_id', location='json')

# From file uploads
parser.add_argument('picture', location='files')
```

- 也可指明多个位置

```
parser.add_argument('text', location=['headers', 'json'])
```

toutiao-flask-v5.06 ©2019.8 yaoxiangyu@itcast.cn be based on toutiao-flask-v5.01 ©2019.5 kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 18:37:29

关于响应处理

[TOC]

- 1. 序列化数据
  - 1.1 步骤
  - 1.2 代码实现
- 2. 定制返回的JSON格式
  - 2.1 需求
  - 2.2 解决
  - 2.3 完整代码如下

1. 序列化数据

> Flask-RESTful 提供了marshal工具，用来帮助我们将数据序列化为特定格式的字典数据，以便作为视图的返回值。

1.1 步骤

> - step 1. 定义要返回数据格式，dict
> - step 2. 在具体的视图函数中返回序列化的数据
> - 对视图函数使用 `marshal_with` 装饰器
> - `@marshal_with(step1定义的数据格式, envelope='返回数据的外层包裹字段')`
> - 在视图函数中使用 `marshal` 函数
> - `marshal(实例化的数据类, step1定义的数据格式, envelope='返回数据的外层包裹字段')`

1.2 代码实现

```
from flask import Flask
from flask_restful import fields, marshal, marshal_with, Resource, Api

# 用来模拟要返回的数据对象的类
class User(object):
    def __init__(self, user_id, name, age):
        self.user_id = user_id
        self.name = name
        self.age = age

# TODO 定义要返回数据格式
resoure_fields = {
```

```
        'user_id': fields.Integer,
        'name': fields.String
    }

class Demo1Resource(Resource):
    # TODO 使用装饰器marshal_with让视图函数返回序列化的数据
    # envelope参数规定了返回数据的外层包裹字段
    @marshal_with(resoure_fields, envelope='data1')
    def get(self):
        # 直接返回实例化的具体的数据对象
        return User(1, 'itcast', 12)

class Demo2Resource(Resource):
    def get(self):
        user = User(1, 'itcast', 12)
        # TODO 使用marshal函数返回序列化的数据
        return marshal(user, resoure_fields, envelope='data2')

app = Flask(__name__)
# 创建api对象，加载app对象
api = Api(app)
# 添加类视图
api.add_resource(Demo1Resource, '/1')
api.add_resource(Demo2Resource, '/2')

if __name__ == '__main__':
    app.run(debug=True)
```

2. 定制返回的JSON格式

2.1 需求

- 想要接口返回的JSON数据具有如下统一的格式

  ```
  {"message": "描述信息", "data": {要返回的具体数据}}
  ```

  - 在接口处理正常的情况下， message返回ok即可，但每个接口只返回要返回的具体数据

  ```
  class DemoResource(Resource):
        def get(self):
            return {'user_id':1, 'name': 'itcast'}
  ```

  - marshal函数或marshal_with装饰器，都只能通过envelope参数控制外层字段，那上述需求如何满足呢？

2.2 解决

> - Flask-RESTful的Api对象提供了一个 `representation` 的装饰器，允许定制返回数据的呈现格式，该装饰器使用方法如下：
>
> ```
> # 伪代码
> api = flask_restful.Api(flask.Flask(__name__))
> @api.representation('application/json')
> def handle_json(data, code, headers):
>  # TODO 此处添加自定义处理
>  return resp
> ```
>
> - 从 `flask_restful.representations.json` 中复制 `output_json` 函数，改写并添加 `representation` 装饰器

2.3 完整代码如下

> ```
> from six import PY3
> from json import dumps
> from flask import Flask, make_response, current_app
> from flask_restful import fields, marshal, marshal_with, Resource, Api
>
> # 用来模拟要返回的数据对象的类
> class User(object):
>  def __init__(self, user_id, name, age):
>  self.user_id = user_id
>  self.name = name
>  self.age = age
>
> # TODO 定义要返回数据格式
> resoure_fields = {
>  'user_id': fields.Integer,
>  'name': fields.String
>  }
>
> class Demo1Resource(Resource):
>  # TODO 使用装饰器marshal_with让视图函数返回序列化的数据
>  # envelope参数规定了返回数据的外层包裹字段
>  @marshal_with(resoure_fields)
>  def get(self):
>  # 直接返回实例化的具体的数据对象
>  return User(1, 'itcast', 12)
>
> class Demo2Resource(Resource):
>  def get(self):
>  user = User(1, 'itcast', 12)
> ```

```
        # TODO 使用marshal函数返回序列化的数据
        return marshal(user, resoure_fields)

app = Flask(__name__)
# 创建api对象，加载app对象
api = Api(app)
# 添加类视图
api.add_resource(Demo1Resource, '/1')
api.add_resource(Demo2Resource, '/2')

# TODO 1. 自定义返回数据的格式：重写output_json函数，该函数位置：flask_restful.represent
# TODO 2. Flask-RESTful的Api对象提供了一个representation装饰器，允许定制返回数据的呈现格
# TODO 3. 传入参数声明：只要是返回的是'application/json'类型的数据，就执行重写的output_js
# from flask_restful.representations.json import output_json
@api.representation('application/json')
def output_json(data, code, headers=None):
    """Makes a Flask response with a JSON encoded body"""

    # 此处为自己添加***************
    if 'message' not in data:
        data = {
            'message': 'OK',
            'data': data
        }
    # *************************

    settings = current_app.config.get('RESTFUL_JSON', {})

    # If we're in debug mode, and the indent is not set, we set it to a
    # reasonable value here.  Note that this won't override any existing value
    # that was set.  We also set the "sort_keys" value.
    if current_app.debug:
        settings.setdefault('indent', 4)
        settings.setdefault('sort_keys', not PY3)

    # always end the json dumps with a new line
    # see https://github.com/mitsuhiko/flask/pull/1262
    dumped = dumps(data, **settings) + "\n"

    resp = make_response(dumped, code)
    resp.headers.extend(headers or {})
    return resp

if __name__ == '__main__':
    app.run(debug=True)
```

kongdehai@itcast.cn all right reserved，powered by Gitbook该文件修订时间： 2019-08-19 18:40:57